Computational Optimization & Applications

Siju Swamy

2025-11-17

Table of contents

Course Vision & Context

The Optimization Revolution in Computational Sciences

In an era dominated by Artificial Intelligence, Machine Learning, and Data Science, *optimization forms the fundamental backbone* that powers intelligent decision-making systems. From recommending your next movie to orchestrating global supply chains, from training deep neural networks to scheduling autonomous vehicles—optimization algorithms are the invisible engines driving technological progress.

This course positions you at the intersection of *mathematical theory* and *computational practice*, equipping you with both the conceptual understanding and hands-on skills to design, implement, and deploy optimization solutions for real-world challenges.

Course Syllabus (48 Hours / 12 Weeks)

Course Overview

Course Code: 20MAT382

Course Name: Computational Optimization and Applications

Duration: 12 Weeks (4 hours/week)

Credits: 4

Total Marks: 150 (Internal: 70 + External: 80)

Intensive Learning Approach

Accelerated project-based curriculum focusing on core optimization concepts with immediate practical application through integrated micro-projects.

Course Objectives

S.No	COURSE OBJECTIVES
1	To gain a comprehensive understanding of optimization concepts and their real-world relevance, emphasizing Python as a practical tool for optimization.
2	To develop proficiency in Python for optimization, including formulating and solving linear programming problems, implementing nonlinear optimization and analysing optimization solutions.
3	To acquire skills in project planning and optimization techniques using Python.
4	To master optimization techniques in the context of machine learning, including Gradient Descent, Stochastic Gradient Descent, and various optimization algorithms, all implemented in Python.
5	To apply optimization knowledge and Python skills to solve combinatorial and graph-based problems, while also considering the ethical aspects of optimization in engineering, logistics, and decision-making.

Course Outcomes

At the end of the course students will be able to:

CO Code	COURSE OUTCOMES	REVISED BLOOM'S TAXONOMY LEVEL
CO1	Demonstrate a thorough understanding of optimization concepts, problem types, and their real-world applications.	3
CO2	Formulate and solve linear programming problems using Python.	3
CO3	Implement nonlinear optimization algorithms, and analyse optimization solutions using Python.	2
CO4	Develop practical project planning skills and proficiency in applying heuristic algorithms to real-world scenarios.	2
CO5	Demonstrate mastery of optimization in Machine Learning, with the ability to apply Gradient Descent, Stochastic Gradient Descent.	3

Core Competencies

- Formulate real-world problems as mathematical optimization models

- Implement optimization algorithms in Python using industry tools
- Analyze and validate optimization solutions
- Develop end-to-end optimization systems for practical applications

Assessment Plan (70 Marks Internal)

Continuous Evaluation (70 Marks)

A. Theory Components (20 Marks)

- **Internal Exam 1**: 10 marks (Week 6)
- **Internal Exam 2**: 10 marks (Week 12)

B. Practical Components (50 Marks)

- **Assignments/Micro Projects**: 15 marks (3 projects × 5 marks each)
- **Lab Exams**: 10 marks (2 exams × 5 marks each)
- **Day-to-Day Lab Work**: 15 marks
- **Attendance**: 10 marks

External Examination (80 Marks)

- **End Semester Theory Exam**: 80 marks

12-Week Delivery Plan (4 Hours/Week)

Phase 1: Foundation & Linear Methods (Weeks 1-4)

Week 1: Optimization Fundamentals & Python Setup (4 hours)

- **Theory (2h)**: Optimization concepts, problem classification, LP formulation
- **Lab (2h)**: `Python` environment setup, `PuLP` introduction
- **Lab Work**: Basic LP implementation (1 mark)
- **Micro-Project 1 Launch**: Campus facility location problem

Week 2: Linear Programming & Solution Methods (4 hours)

- **Theory (2h)**: Graphical method, Simplex algorithm
- **Lab (2h)**: `PuLP` implementation, constraint handling
- **Lab Work**: Complex constraint implementation (1 mark)
- **Attendance**: Week 1-2 (2 marks)

Week 3: Advanced LP & Real Applications (4 hours)

- **Theory (1h)**: Sensitivity analysis, duality
- **Lab (3h)**: Transportation problems, case studies
- **Lab Work**: Transportation problem solution (1 mark)
- **Micro-Project 1 Due**: Submission (5 marks)

Week 4: Nonlinear Optimization Foundations (4 hours)

- **Theory (2h)**: Unconstrained optimization, Golden Section
- **Lab (2h)**: `SciPy` optimization, function minimization
- **Lab Work**: Nonlinear solver implementation (1 mark)
- **Attendance**: Week 3-4 (2 marks)

Phase 2: Constrained & Combinatorial Methods (Weeks 5-8)

Week 5: Constrained Optimization (4 hours)

- **Theory (2h)**: KKT conditions, constraint handling
- **Lab (2h)**: Constrained NLP implementation
- **Lab Work**: KKT condition implementation (1 mark)
- **Micro-Project 2 Launch**: Nonlinear cost optimization
- **Lab Exam 1**: Basic LP/NLP implementation (5 marks)

Week 6: Project Planning & Heuristics (4 hours)

- **Theory (1h)**: CPM/PERT fundamentals
- **Lab (3h)**: Project scheduling, greedy algorithms
- **Internal Exam 1**: Theory assessment (10 marks)
- **Lab Work**: Project scheduling implementation (1 mark)

Week 7: Graph Algorithms I (4 hours)

- **Theory (1h)**: Graph theory, shortest path concepts
- **Lab (3h)**: NetworkX implementation, Dijkstra's algorithm
- **Lab Work**: Shortest path implementation (1 mark)
- **Micro-Project 2 Due**: Submission (5 marks)
- **Attendance**: Week 5-7 (2 marks)

Week 8: Graph Algorithms II (4 hours)

- **Theory (1h)**: MST, network flows, TSP overview
- **Lab (3h)**: Advanced graph algorithms
- **Lab Work**: MST implementation (1 mark)
- **Micro-Project 3 Launch**: Routing optimization

Phase 3: Advanced Applications & Integration (Weeks 9-12)

Week 9: Machine Learning Optimization I (4 hours)

- **Theory (2h)**: Gradient Descent, SGD, optimization in ML
- **Lab (2h)**: Basic GD implementation
- **Lab Work**: Gradient descent implementation (1 mark)
- **Attendance**: Week 8-9 (2 marks)

Week 10: Machine Learning Optimization II (4 hours)

- **Theory (1h)**: Advanced optimizers, neural networks
- **Lab (3h)**: TensorFlow/PyTorch optimization
- **Lab Work**: Advanced optimizer implementation (1 mark)
- **Lab Exam 2**: Graph and ML optimization (5 marks)

Week 11: Integrated Applications (4 hours)

- **Workshop (4h)**: Comprehensive system implementation
- **Lab Work**: Integrated system development (2 marks)
- **Micro-Project 3 Due**: Submission (5 marks)

Week 12: Review & Final Assessment (4 hours)

- **Internal Exam 2**: Theory assessment (10 marks)
- **Course Review**: Comprehensive concepts revision
- **Lab Work**: Final implementation polish (1 mark)
- **Attendance**: Week 10-12 (2 marks)

Thematic Project: Campus City Supply Chain

Micro-Projects (15 Marks Total)

Micro-Project 1: Basic LP Implementation (5 marks)

- **Timeline**: Week 1-3
- **Scope**: 6 facilities, 3 warehouses, linear costs
- **Assessment**: Model correctness (2), Code quality (2), Documentation (1)

Micro-Project 2: Nonlinear Optimization (5 marks)

- **Timeline**: Week 4-7
- **Scope**: Enhanced cost models, KKT conditions
- **Assessment**: Algorithm implementation (2), Analysis (2), Validation (1)

Micro-Project 3: Graph & Network Optimization (5 marks)

- **Timeline**: Week 8-11
- **Scope**: Routing, shortest paths, resource allocation
- **Assessment**: System design (2), Performance (2), Documentation (1)

Detailed Mark Distribution

Day-to-Day Lab Work (15 Marks)

- **Weekly Implementation Tasks**: 12 marks (1 mark × 12 weeks)
- **Integrated System Development**: 3 marks (Week 11)

Attendance (10 Marks)

- **Weekly Attendance**: 2 marks per 3-week block
- **Full Attendance Bonus**: 2 marks for 100% attendance

Lab Exams (10 Marks)

- **Lab Exam 1** (Week 5): Basic LP/NLP implementation (5 marks)
- **Lab Exam 2** (Week 10): Graph and ML optimization (5 marks)

Internal Exams (20 Marks)

- **Internal Exam 1** (Week 6): Modules 1-2 theory (10 marks)
- **Internal Exam 2** (Week 11): Modules 3-4 theory (10 marks)

Learning Outcomes Mapping

Theory Outcomes (Internal Exams + External)

- Formulate optimization problems mathematically
- Understand algorithm properties and convergence
- Analyze problem structures and solution methods

Practical Outcomes (Lab Work + Projects)

- Implement optimization algorithms in Python
- Develop end-to-end optimization systems
- Validate and analyze optimization results
- Create professional documentation and visualizations

Grading Rubrics

Micro-Projects (5 marks each)

- **Excellent (5)**: Flawless implementation with advanced features
- **Very Good (4)**: Correct implementation with good documentation
- **Good (3)**: Basic functionality with minor issues
- **Satisfactory (2)**: Meets minimum requirements
- **Poor (1)**: Significant functionality missing

Lab Work (Weekly 1 mark)

- **Complete (1)**: Task fully implemented and demonstrated
- **Partial (0.5)**: Basic implementation with issues
- **Incomplete (0)**: Task not attempted or completely non-functional

Lab Exams (5 marks each)

- **Algorithm Implementation**: 2 marks
- **Problem Solving**: 2 marks
- **Code Quality**: 1 mark

Success Strategy

Maximizing Internal Marks

- **Consistent Attendance**: 10 marks easily achievable
- **Regular Lab Work**: 15 marks through weekly completion
- **Quality Projects**: 15 marks with careful implementation
- **Lab Exam Preparation**: 10 marks with practice
- **Internal Exam Focus**: 20 marks through concept mastery

External Exam Preparation (80 Marks)

- Comprehensive theory coverage from all modules
- Problem-solving practice with various optimization types
- Mathematical formulation skills
- Algorithm analysis and comparison

Weekly Preparation Guide

Before Each Week

- Review weekly objectives and deliverables
- Prepare development environment
- Read theoretical concepts in advance

During Each Week

- Attend all sessions (critical for attendance marks)
- Complete lab work during sessions
- Start micro-projects early
- Seek clarification immediately

After Each Week

- Submit all lab work promptly
- Review concepts for internal exams
- Prepare for upcoming assessments
- Maintain code repository

This assessment-focused syllabus ensures students can maximize their 70 internal marks through consistent performance while preparing comprehensively for the 80-mark external examination. The structured approach balances theoretical understanding with practical implementation skills.

1 Introduction to Computational Optimization and Applications

```
Welcome to Computational Optimization & Applications
Where Mathematics Meets Computational Intelligence
Bridging Theory and Practice in the AI Revolution
```

1.1 Why Optimization Matters Now More Than Ever

The exponential growth in data complexity and computational requirements has transformed optimization from a theoretical discipline to an essential toolkit for every computer scientist and data professional. Consider these real-world contexts:

- **AI Systems**: Training neural networks is essentially an optimization process (Gradient Descent)
- **Operations Research**: Logistics, scheduling, and resource allocation drive billion-dollar efficiencies

- **Data Science**: Model selection, hyperparameter tuning, and feature engineering are optimization problems
- **Autonomous Systems**: Path planning, control systems, and decision-making rely on optimization algorithms
- **Quantum Computing**: Many quantum algorithms are designed to solve optimization problems more efficiently

1.2 Programme Objectives & Learning Outcomes

1.2.1 Core Educational Mission

This minor programme is designed to bridge the critical gap between theoretical optimization mathematics and practical computational implementation. Upon successful completion, you will be able to:

Domain	Learning Outcomes
Theoretical Foundation	• Formulate real-world problems as mathematical optimization models • Understand optimality conditions and convergence properties • Analyze problem structures to select appropriate solution methods
Computational Skills	• Implement classical and modern optimization algorithms in Python • Utilize industry-standard optimization libraries and frameworks • Develop end-to-end optimization pipelines for practical applications
AI/ML Integration	• Understand optimization's role in training machine learning models • Implement gradient-based methods for neural network optimization • Apply optimization to hyperparameter tuning and model selection
Problem-Solving	• Design optimization solutions for complex, multi-objective problems • Evaluate solution quality and algorithm performance • Communicate optimization insights to technical and non-technical stakeholders

1.2.2 The "Smart City Logistics" Experiential Thread

Throughout this course, we'll employ a *continuous practical thread*: optimizing logistics and operations for a smart city ecosystem. This narrative provides:

- **Real-world context** for theoretical concepts
- **Progressive complexity** as we advance through modules
- **Portfolio-building** implementation experience
- **Industry-relevant** problem-solving skills

Smart City Optimization Journey:

- **Module 1**: Warehouse Location (Linear Programming)
- **Module 2**: Fuel Cost Optimization (Nonlinear Programming)
- **Module 3**: Delivery Routing (Graph Algorithms)
- **Module 4**: Dynamic Scheduling (Heuristic Methods)
- **Module 5**: Demand Prediction (ML Integration)

1.3 Course Roadmap & Syllabus Integration

1.3.1 Module Progression: From Foundations to Frontiers

Our journey through computational optimization is strategically sequenced to build from fundamental principles to advanced applications:

1.3.1.1 Foundation Phase: Mathematical Underpinnings

- **Module I**: Linear Programming & Formulation Skills
- **Module II**: Nonlinear Optimization & Constraint Handling

1.3.1.2 Advanced Phase: Algorithmic Thinking

- **Module III**: Project Planning & Heuristic Methods
- **Module IV**: Combinatorial & Graph Optimization

1.3.1.3 Integration Phase: AI/ML Applications

- **Module V**: Gradient Methods & Machine Learning Optimization

1.3.2 Assessment Strategy: Theory Meets Practice

To ensure comprehensive understanding and skill development, assessment integrates both theoretical knowledge and practical implementation:

- **Series Examinations**: Test conceptual understanding and problem formulation
- **Practical Assignments/ Micro Project**: Evaluate implementation skills and computational thinking

- **Final Project**: Assess integrated problem-solving and solution design
- **Continuous Evaluation**: Monitor progress through micro-projects and code reviews

1.4 Optimization in the AI/ML Ecosystem

1.4.1 The Central Role in Machine Learning

Optimization isn't just adjacent to machine learning—**it is machine learning**. The entire process of training machine learning models revolves around optimization principles:

- **Loss Minimization**: Finding model parameters that minimize prediction error
- **Convergence Analysis**: Understanding when and how algorithms reach optimal solutions
- **Regularization**: Balancing model complexity with performance through constrained optimization
- **Hyperparameter Tuning**: Optimizing the optimization process itself

1.4.2 Emerging Trends & Future Directions

The field of optimization is rapidly evolving, driven by advances in:

- **Large-Scale Optimization**: Methods for billion-parameter models in deep learning
- **Automated Optimization**: AutoML and neural architecture search
- **Quantum Optimization**: Quantum annealing and hybrid quantum-classical algorithms

- **Federated Optimization**: Privacy-preserving distributed learning
- **Multi-Objective Optimization**: Pareto optimization for conflicting objectives
- **Explainable Optimization**: Interpretable and transparent optimization processes

1.5 Technical Ecosystem & Tools

1.5.1 Why Python for Optimization?

Python has emerged as the **lingua franca** for computational optimization due to:

- **Rich Ecosystem**: Comprehensive libraries for every optimization paradigm
- **AI/ML Integration**: Seamless connection with machine learning frameworks
- **Performance**: C/Fortran-backed numerical computing with Python simplicity
- **Community**: Vibrant ecosystem with continuous algorithm development
- **Industry Adoption**: Widely used in both academia and industry

1.5.2 Core Toolchain

Throughout this course, we'll work with industry-standard tools:

Our Computational Optimization Stack:

Category	Tools
Numerical Computing	NumPy, SciPy
Linear Programming	PuLP, CVXPY
Machine Learning	Scikit-learn, TensorFlow, PyTorch
Graph Algorithms	NetworkX
Visualization	Matplotlib, Plotly, Seaborn
Development	Jupyter, VSCode, Git

1.6 Getting Started: Your Learning Journey

1.6.1 Prerequisites & Preparation

To succeed in this course, you should have:

- **Programming Fundamentals**: Basic Python proficiency
- **Mathematical Background**: Linear algebra and calculus foundations

- **Computational Mindset**: Willingness to experiment and debug
- **Problem-Solving Attitude**: Persistence through challenging concepts
- **Curiosity and Creativity**: Interest in exploring multiple solution approaches

1.6.2 How to Maximize Your Learning

1. **Engage Actively**: Don't just read—implement every concept in code
2. **Think Critically**: Question why certain methods work better for specific problems
3. **Experiment Freely**: Modify parameters, break code, and learn from failures
4. **Connect Concepts**: Relate theoretical principles to practical implementations
5. **Build Portfolio**: Document your work for future career opportunities
6. **Collaborate Effectively**: Learn from peers through code reviews and discussions
7. **Stay Updated**: Follow recent developments in optimization research

1.6.3 Course Structure and Expectations

This course is designed as a **blended learning experience** combining:

- **Theoretical Foundations**: Mathematical principles and algorithm concepts
- **Practical Implementation**: Hands-on coding exercises and projects
- **Real-World Applications**: Industry-relevant case studies and problems
- **Assessment and Feedback**: Continuous evaluation and improvement

1.7 Welcome to the Journey

You are beginning a journey into one of the most fundamental and powerful domains of computer science and artificial intelligence. The skills you develop here will serve as a foundation for advanced work in machine learning, operations research, data science, and algorithmic design.

As we progress through the modules, remember that each concept builds toward a comprehensive understanding of how to make computers not just compute, but **optimize**—transforming them from calculators into intelligent decision-makers.

The journey through computational optimization is challenging but immensely rewarding. You'll gain not just technical skills, but a new way of thinking about problem-solving that will serve you throughout your career in technology.

> **"Optimization is the science of better. In a world of limited resources and unlimited wants, optimization provides the mathematical foundation for making the best possible decisions."**

---

2 Module 1: Basics of Optimization and Linear Programming

2.1 Module Overview

2.1.1 Learning Objectives

- Understand fundamental optimization concepts and problem classification
- Formulate real-world problems as Linear Programming models
- Solve LP problems using graphical and computational methods
- Implement LP solutions in Python for practical applications
- Apply optimization thinking to facility location problems

2.1.2 Smart City Context: Warehouse Location Challenge

In our Smart City Logistics project, we face a critical business decision: *where to locate new distribution warehouses* to minimize transportation costs while serving all city facilities efficiently. This module provides the mathematical foundation to solve this strategic problem.

2.2 Foundations of Optimization

2.2.1 What is Optimization?

Optimization is the science of finding the *best possible solution* from all feasible alternatives under given constraints. In computational terms, it involves:

- **Decision Variables**: Quantities we can control (e.g., warehouse locations, shipment quantities)
- **Objective Function**: What we want to maximize or minimize (e.g., total transportation cost)
- **Constraints**: Limitations and requirements (e.g., budget, capacity, demand)

2.2.2 Optimization Problem Classification

Problem Type	Characteristics	Smart City Example
Linear Programming	Linear objective and constraints	Warehouse location with fixed costs
Nonlinear Programming	Nonlinear relationships	Fuel costs that increase with distance
Constrained Optimization	With limitations	Budget constraints on construction
Unconstrained Optimization	No limitations	Theoretical ideal locations
Discrete Optimization	Integer decisions	Yes/no decisions for locations
Continuous Optimization	Real-valued decisions	Precise coordinates for facilities

2.2.3 Real-World Applications in CSE

- **Resource Allocation**: CPU time, memory allocation in operating systems
- **Network Optimization**: Internet routing, data flow optimization
- **Machine Learning**: Model training via loss function minimization
- **Database Systems**: Query optimization and indexing
- **Computer Graphics**: Rendering optimization and path tracing

2.3 Basic Python for Optimization

2.3.1 Essential Python Libraries Setup

```
# Core optimization stack installation
# pip install numpy scipy matplotlib pulp pandas jupyter
```

Key `Python` Libraries Required for Computational Part

- `NumPy`: Numerical computing and array operations
- `SciPy`: Scientific computing and optimization algorithms
- `Matplotlib`: Data visualization and result plotting

- `PuLP`: Linear programming interface
- `Pandas`: Data manipulation and analysis

2.3.2 Python Fundamentals for Optimization

```python
# Essential operations we'll use frequently
import numpy as np
import matplotlib.pyplot as plt

# Array operations for constraint matrices
coefficients = np.array([[2, 1], [1, 3], [4, 2]])

# Function definitions for objectives
def transportation_cost(warehouses, facilities):
    return np.sum(warehouses * facilities)

# Data handling for problem parameters
demand_data = {'Hospital': 50, 'School': 30, 'Mall': 80}
```

2.4 Linear Programming Fundamentals

2.4.1 Mathematical Definition of Linear Programming

A Linear Programming (LP) problem can be formally defined as:

Objective Function:

$$\text{Optimize } Z = c_1x_1 + c_2x_2 + \cdots + c_nx_n$$

Subject to Constraints:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &\leq b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &\leq b_2 \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &\leq b_m \end{aligned}$$

Non-negativity Conditions:

$$x_1, x_2, \ldots, x_n \geq 0$$

Where:

- Z is the objective function to be optimized (maximized or minimized)
- $x_1, x_2, \ldots, x_n$ are the decision variables
- $c_1, c_2, \ldots, c_n$ are the coefficients of the objective function
- a_{ij} are the technological coefficients
- $b_1, b_2, \ldots, b_m$ are the right-hand side constants

2.4.2 Standard Forms of Linear Programming

2.4.2.1 Canonical Form (Maximization)

$$\begin{aligned} \text{Maximize } & Z = \mathbf{c}^T\mathbf{x} \\ \text{Subject to } & \mathbf{A}\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

2.4.2.2 Standard Form (Maximization)

$$\begin{aligned} \text{Maximize } & Z = \mathbf{c}^T\mathbf{x} \\ \text{Subject to } & \mathbf{A}\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

2.4.3 Key Concepts in Linear Programming

2.4.3.1 Feasible Region

The set of all points that satisfy all constraints simultaneously:

$$F = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$$

2.4.3.2 Optimal Solution

A point $\mathbf{x}^*$ in the feasible region that gives the best value of the objective function.

2.4.3.3 Corner Point (Extreme Point)

A point in the feasible region that cannot be expressed as a convex combination of two other distinct points in the region.

2.4.4 Smart City Case: Warehouse Location Formulation

2.4.4.1 Problem Statement

We need to choose locations for 2 warehouses from 4 potential sites to serve 3 major facilities while minimizing total costs.

2.4.4.2 Decision Variables

Let: - x_{ij}: Amount shipped from warehouse i to facility j - y_i: Binary variable (1 if warehouse i is built, 0 otherwise)

Where: - $i = 1, 2, 3, 4$ (warehouse sites) - $j = 1, 2, 3$ (facilities: Hospital, Mall, School)

2.4.4.3 Objective Function

Minimize total cost:

$$\text{Minimize } Z = \sum_{i=1}^{4} \sum_{j=1}^{3} c_{ij} x_{ij} + \sum_{i=1}^{4} f_i y_i$$

Where: - c_{ij}: Transportation cost per unit from warehouse i to facility j - f_i: Fixed construction cost for warehouse i

2.4.4.4 Constraints

Demand Constraints (Each facility's demand must be met):

$$\sum_{i=1}^{4} x_{ij} = d_j \quad \text{for } j = 1, 2, 3$$

Where d_j is the demand of facility j.

Capacity Constraints (Warehouse capacity cannot be exceeded):

$$\sum_{j=1}^{3} x_{ij} \leq C_i y_i \quad \text{for } i = 1, 2, 3, 4$$

Where C_i is the capacity of warehouse i.

Budget Constraint:

$$\sum_{i=1}^{4} f_i y_i \leq B$$

Where B is the total budget available.

Warehouse Selection Constraint (Select exactly 2 warehouses):

$$\sum_{i=1}^{4} y_i = 2$$

Non-negativity and Binary Requirements:

$$x_{ij} \geq 0 \quad \text{for all } i, j$$

$$y_i \in \{0, 1\} \quad \text{for all } i$$

2.4.5 Graphical Method for Two-Variable Problems

2.4.5.1 Step-by-Step Procedure

1. **Plot Constraints**: Convert each inequality to equality and plot the line
2. **Identify Feasible Region**: Determine which side of each line satisfies the inequality
3. **Find Corner Points**: Identify intersection points of constraint boundaries
4. **Evaluate Objective Function**: Calculate objective value at each corner point
5. **Identify Optimal Solution**: Select the corner point with best objective value

2.4.5.2 Example: Simple Warehouse Problem

Consider a simplified version with 2 warehouses and 1 facility:

Problem:

$$\begin{aligned} \text{Maximize } Z &= 40x_1 + 30x_2 \\ \text{Subject to } 2x_1 + x_2 &\leq 100 \\ x_1 + 3x_2 &\leq 90 \\ x_1 \geq 0, x_2 &\geq 0 \end{aligned}$$

Step 1: Plot Constraints - Constraint 1: $2x_1 + x_2 = 100 \rightarrow$ Line through (0,100) and (50,0)
- Constraint 2: $x_1 + 3x_2 = 90 \rightarrow$ Line through (0,30) and (90,0)

Step 2: Identify Corner Points - Intersection of axes: (0,0), (0,30), (50,0) - Intersection of constraints: Solve:

$$2x_1 + x_2 = 100$$
$$x_1 + 3x_2 = 90$$

Solution: $x_1 = 42, x_2 = 16$

Step 3: Evaluate Objective Function

- At (0,0): $Z = 0$
- At (0,30): $Z = 900$
- At (50,0): $Z = 2000$
- At (42,16): $Z = 40 \times 42 + 30 \times 16 = 2160$

Optimal Solution: $x_1 = 42, x_2 = 16$ with $Z = 2160$

2.4.6 Simplex Method Theory

2.4.6.1 Fundamental Theorem of Linear Programming

If an LP problem has an optimal solution, then there exists at least one corner point of the feasible region that is optimal.

2.4.6.2 Simplex Algorithm Steps

1. **Convert to Standard Form**
    - Convert inequalities to equations using slack variables
    - Ensure all variables are non-negative
    - Convert minimization to maximization if needed
2. **Initial Basic Feasible Solution**
    - Identify basic variables (variables with coefficient 1 in one equation and 0 in others)
    - Set non-basic variables to zero
3. **Optimality Test**
    - Calculate reduced costs for non-basic variables
    - If all reduced costs ≤ 0 (for maximization), current solution is optimal
4. **Pivot Column Selection**
    - Choose non-basic variable with most positive reduced cost (for maximization)

5. **Pivot Row Selection**
    - Use minimum ratio test: $\min\left\{\frac{b_i}{a_{ij}} : a_{ij} > 0\right\}$
6. **Pivot Operation**
    - Make pivot element equal to 1
    - Make other elements in pivot column equal to 0
7. **Repeat** until optimality condition is satisfied

2.4.6.3 Simplex Tableau Format

The simplex method uses a tableau to organize calculations:

Basic Var	x_1	x_2	s_1	s_2	RHS
s_1	2	1	1	0	100
s_2	1	3	0	1	90
Z	-40	-30	0	0	0

Where s_1, s_2 are slack variables.

2.5 Types of Linear Programming Solutions

When we feed a problem into a solver (like the Simplex algorithm), we usually expect a single, perfect answer. But algorithms are strict logicians. If we ask for the impossible or the infinite, they will tell us exactly that. In Linear Programming, there are four distinct outcomes.

2.5.1 Unique Optimal Solution

The "Ideal" Scenario

This is the standard result. Geometrically, the feasible region is a polygon, and the Objective Function (the "slope" of profit or cost) cuts through the region such that it touches exactly **one** corner point (vertex) last.

- **Mathematical Context:** The slope of the objective function does *not* match the slope of any binding constraint.
- **Engineering Implication:** There is exactly one best way to allocate your resources. Any deviation from this plan results in lower profit or higher cost.

2.5.2 Multiple Optimal Solutions (Alternative Optima)

The "Flexibility" Scenario

Sometimes, you might get a result where the solver says: *"You can choose Point A ($x = 2, y = 5$) or Point B ($x = 4, y = 3$), and the profit is exactly the same."*

- **Why this happens:** The slope of your Objective Function is **parallel** to one of the binding constraint lines. Instead of touching a single corner, the objective function lies flat against an entire edge of the feasible region.
- **Engineering Implication:** This is actually great news for a designer! It means you have **flexibility**. Mathematically, the cost is the same, but maybe Plan A is politically easier to implement than Plan B. You can choose based on secondary factors (like aesthetics or risk) without losing optimality.

2.5.3 Unbounded Solution

The "Too Good to Be True" Scenario

Imagine writing a program to maximize profit, and the output is `Infinity`. While exciting, this is invariably a **modeling error**.

- **Why this happens:** The feasible region is not closed (it extends infinitely in one direction), and you are trying to maximize in that direction. You have forgotten a constraint.
- **Real-world Example:** "Maximize production." If you don't add a constraint for "Available Raw Materials" or "Factory Capacity," the math assumes you can produce infinite units.
- **Mathematical Check:** In the Simplex tableau, this occurs when a variable can enter the basis, but no variable leaves (no positive ratio in the pivot test).

2.5.4 Infeasible Problem

The "Impossible" Scenario

This is the most common frustration in real-world engineering. You hit "Run," and the solver returns an error.

- **Why this happens:** The constraints are contradictory. There is no overlap in the shaded regions. The Feasible Region is empty ($F = \emptyset$).
- **Real-world Example:** "Build a warehouse that is within 5km of the city center ($x \leq 5$) AND at least 10km away from residential zones ($x \geq 10$)." It is mathematically impossible to satisfy both.
- **Debugging:** You must relax (loosen) one or more constraints to find a solution.

2.6 Duality in Linear Programming

2.6.1 The "Mirror Image" of Optimization

Every Linear Programming problem (which we call the **Primal**) has a ghostly twin brother called the **Dual**.

If the **Primal** problem asks: *"How do I mix these ingredients to make the most profit?"* The **Dual** problem asks: *"How much are these ingredients actually worth to me?"*

2.6.2 Why do Computer Scientists care?

1. **Computational Efficiency:** Sometimes the Primal problem has 100,000 constraints and is slow to solve. The Dual might have only 100 constraints and is much faster. Solving the Dual gives you the exact same optimal value (Z^*) as the Primal.
2. **Shadow Prices (Economic Interpretation):** The solution values of the Dual variables ($y_1, y_2...$) tell you the **Shadow Price** of your resources.
    - *Insight:* If the shadow price of "Machine Hours" is \$50, it means acquiring **one extra hour** of machine time will increase your total profit by exactly \$50. This tells managers where to invest!

2.6.3 The Mathematical Transformation

We convert a Primal to a Dual by flipping the matrix: * **Maximization** becomes **Minimization**. * **Right-Hand Side (b)** constants of Primal become **Objective Coefficients (c)** of Dual. * **Constraint Matrix (A)** is transposed (A^T).

2.6.4 Key Theorems

- **Weak Duality Theorem:** The value of *any* feasible solution to the Maximization problem is always $\leq$ the value of *any* feasible solution to the Minimization problem. They bound each other.
- **Strong Duality Theorem:** If the Primal has an optimal solution, the Dual has an optimal solution, and **their Objective Values are equal** ($Z_{primal} = W_{dual}$).

Every LP problem (primal) has a corresponding dual problem:

Primal Problem:

$$\begin{aligned} \text{Maximize } Z &= \mathbf{c}^T\mathbf{x} \\ \text{Subject to } \mathbf{A}&\mathbf{x} \leq \mathbf{b} \\ \mathbf{x} &\geq \mathbf{0} \end{aligned}$$

Dual Problem:

$$\begin{aligned} &\text{Minimize } W = \mathbf{b}^T\mathbf{y} \\ &\text{Subject to } \mathbf{A}^T\mathbf{y} \geq \mathbf{c} \\ &\mathbf{y} \geq \mathbf{0} \end{aligned}$$

2.6.4.1 Duality Theorems

1. **Weak Duality Theorem**: For any feasible solution $\mathbf{x}$ of primal and $\mathbf{y}$ of dual:

$$\mathbf{c}^T\mathbf{x} \leq \mathbf{b}^T\mathbf{y}$$

2. **Strong Duality Theorem**: If either primal or dual has an optimal solution, then both have optimal solutions and:

$$\mathbf{c}^T\mathbf{x}^* = \mathbf{b}^T\mathbf{y}^*$$

2.6.5 Sensitivity Analysis

Sensitivity analysis examines how changes in parameters affect the optimal solution:

2.6.5.1 Changes in Objective Function Coefficients

- Range of optimality for each coefficient
- Effect on optimal solution when coefficients change

2.6.5.2 Changes in Right-Hand Side Constants

- Shadow prices (dual variables)
- Range of feasibility for each constraint

2.6.5.3 Changes in Constraint Coefficients

- Effect of adding new variables or constraints
- Impact of changing technological coefficients

2.6.6 Special Cases in Linear Programming

2.6.6.1 Transportation Problems

Special structure where sources supply destinations with minimum transportation cost.

2.6.6.2 Assignment Problems

Special case of transportation problem where each source is assigned to exactly one destination.

2.6.6.3 Network Flow Problems

Optimization problems defined on networks with flow conservation constraints.

2.7 Linear Programming Problems — Graphical Method

Below are **five LPP problems** designed to be solved using the **graphical method**. For each problem:

1. Draw each constraint as a straight line in the x_1–x_2 plane.
2. Identify the feasible region (including $x_1, x_2 \geq 0$).
3. Find all corner (vertex) points of the feasible region.
4. Evaluate the objective Z at each corner to determine the optimum.
5. State the optimal solution and the optimal value $Z^\star$.

Problem 1 — Simple maximization

Maximize

$$Z = 3x_1 + 2x_2$$

Subject to

$$\begin{aligned} x_1 + x_2 &\leq 4, \\ x_1 &\leq 3, \\ x_2 &\leq 2, \\ x_1, x_2 &\geq 0. \end{aligned}$$

Problem 2 — Two binding inequalities

Maximize

$$Z = 5x_1 + 4x_2$$

Subject to

$$\begin{aligned} 2x_1 + x_2 &\leq 8, \\ x_1 + 2x_2 &\leq 8, \\ x_1, x_2 &\geq 0. \end{aligned}$$

Problem 3 — Minimization with intersection corner

Minimize

$$Z = 4x_1 + 6x_2$$

Subject to

$$\begin{aligned} x_1 + x_2 &\geq 4, \\ 2x_1 + x_2 &\geq 6, \\ x_1, x_2 &\geq 0. \end{aligned}$$

Problem 4 — Resource allocation (mix of $\leq$ and $\geq$)

A factory makes two products P_1 and P_2 with profits \$7 and \$5 respectively.

Maximize profit

$$Z = 7x_1 + 5x_2$$

Subject to resource limits

$$\begin{aligned} 3x_1 + 2x_2 &\leq 18 \quad \text{(machine-hours)}, \\ x_1 + 2x_2 &\geq 4 \quad \text{(minimum production constraint)}, \\ x_1, x_2 &\geq 0. \end{aligned}$$

Problem 5 — Problem with a redundant constraint

Maximize

$$Z = 2x_1 + 3x_2$$

Subject to

$$\begin{aligned} x_1 + 4x_2 &\leq 12, \\ 2x_1 + 8x_2 &\leq 24, \\ x_1 + x_2 &\leq 5, \\ x_1, x_2 &\geq 0. \end{aligned}$$

2.8 Practical Implementation with Python

Problem Statement

A Smart City logistics company needs to determine the optimal distribution strategy between two warehouses (Warehouse A and Warehouse B) to minimize total operational costs. The company must decide how many units to ship from each warehouse while considering capacity constraints and transportation costs.

2.8.0.1 Problem Data:

- **Warehouse A**: Transportation cost = $40 per unit
- **Warehouse B**: Transportation cost = $30 per unit
- **Constraint 1**: Minimum demand exeeds 60
- **Constraint 2**: Warehouse A has a maximum capacity of 50
- **Constraint 3**: Warehouse B has a maximum capacity of 40
- **Constraint 4**: Total units from both warehouses cannot exceed 200 (2 units from A + 3 unit from B combination)
- Both warehouses have non-negative shipment quantities

2.8.1 Mathematical Formulation

2.8.1.1 Decision Variables

Let: - $ x_1 $: Number of units to ship from Warehouse A - $ x_2 $: Number of units to ship from Warehouse B

2.8.1.2 Objective Function

Minimize the total transportation cost:

$$\text{Minimize } Z = 40x_1 + 30x_2$$

2.8.2 Complete Linear Programming Model

$$\begin{aligned} \text{Minimize } Z &= 40x_1 + 30x_2 \\ \text{Subject to } x_1 + x_2 &\geq 60 \\ x_1 &\leq 50 \\ x_2 &\leq 40 \\ 2x_1 + 3x_2 &\leq 200 \\ x_1 &\geq 0 \\ x_2 &\geq 0 \end{aligned}$$

Using PuLP for LP Problems

PuLP provides a high-level interface for formulating and solving optimization problems:

```
import pulp

# Create optimization problem
model = pulp.LpProblem("Warehouse_Location", pulp.LpMinimize)

# Define decision variables
x1 = pulp.LpVariable('Warehouse_A', lowBound=0, cat='Continuous')
x2 = pulp.LpVariable('Warehouse_B', lowBound=0, cat='Continuous')

# Define objective function
model += 40*x1 + 30*x2, "Total_Cost"

# Add realistic constraints
model += x1 + x2 >= 60, "Minimum_Demand"
model += x1 <= 50, "Warehouse_A_Capacity"  # Warehouse A max capacity
model += x2 <= 40, "Warehouse_B_Capacity"  # Warehouse B max capacity
model += 2*x1 + 3*x2 <= 200, "Budget_Constraint"  # Combined resource constraint

# Solve the problem
model.solve()

# Print results
print(f"Status: {pulp.LpStatus[model.status]}")
print(f"Optimal Cost: ${pulp.value(model.objective):.2f}")
print(f"Warehouse A: {x1.varValue} units")
print(f"Warehouse B: {x2.varValue} units")
```

```
Status: Optimal
```

```
Optimal Cost: $2000.00
Warehouse A: 20.0 units
Warehouse B: 40.0 units
```

2.9 Linear Programming Problems Collection

2.9.1 Problem 1

A manufacturing company produces two products (A and B) using three machines (M1, M2, M3). The profit per unit is \$50 for product A and \$40 for product B. Machine time requirements and availability are given below. Determine the optimal production quantities to maximize profit.

Machine	Product A (hrs/unit)	Product B (hrs/unit)	Available Hours
M1	2	1	100
M2	1	2	80
M3	1	1	60

Mathematical Formulation

Decision Variables:

- x_1: Units of Product A to produce
- x_2: Units of Product B to produce

Objective Function:

$$\text{Maximize } Z = 50x_1 + 40x_2$$

Constraints:

$$\begin{aligned} 2x_1 + x_2 &\leq 100 \quad \text{(Machine M1)} \\ x_1 + 2x_2 &\leq 80 \quad \text{(Machine M2)} \\ x_1 + x_2 &\leq 60 \quad \text{(Machine M3)} \\ x_1, x_2 &\geq 0 \end{aligned}$$

`Python` Solution

```python
import pulp

# Create the optimization problem
model = pulp.LpProblem("Product_Mix_Optimization", pulp.LpMaximize)

# Decision variables
x1 = pulp.LpVariable('Product_A', lowBound=0, cat='Continuous')
x2 = pulp.LpVariable('Product_B', lowBound=0, cat='Continuous')

# Objective function
model += 50*x1 + 40*x2, "Total_Profit"

# Constraints
model += 2*x1 + x2 <= 100, "Machine_M1"
model += x1 + 2*x2 <= 80, "Machine_M2"
model += x1 + x2 <= 60, "Machine_M3"

# Solve
model.solve()

# Results
print("=== PRODUCT MIX OPTIMIZATION ===")
print(f"Status: {pulp.LpStatus[model.status]}")
print(f"Optimal Profit: ${pulp.value(model.objective):.2f}")
print(f"Product A: {x1.varValue} units")
print(f"Product B: {x2.varValue} units")
print(f"Machine M1 usage: {2*x1.varValue + x2.varValue}/100 hours")
print(f"Machine M2 usage: {x1.varValue + 2*x2.varValue}/80 hours")
print(f"Machine M3 usage: {x1.varValue + x2.varValue}/60 hours")
```

```
=== PRODUCT MIX OPTIMIZATION ===
Status: Optimal
Optimal Profit: $2800.00
Product A: 40.0 units
Product B: 20.0 units
Machine M1 usage: 100.0/100 hours
Machine M2 usage: 80.0/80 hours
Machine M3 usage: 60.0/60 hours
```

2.9.2 Problem 2: Diet Problem

A nutritionist needs to design a minimum-cost diet that meets daily nutritional requirements. Two foods are available with different nutrient contents and costs. Determine the optimal food quantities.

Nutrient	Food 1 (units/kg)	Food 2 (units/kg)	Minimum Daily Requirement
Protein	2	1	8 units
Carbs	1	2	10 units
Fat	1	1	6 units
Cost ($)	3	2	-

Mathematical Formulation

Decision Variables:

- x_1: kg of Food 1 to include in daily diet
- x_2: kg of Food 2 to include in daily diet

Objective Function:

Minimize the total daily cost:

$$\text{Minimize } Z = 3x_1 + 2x_2$$

Constraints:

Protein Requirement:

$$2x_1 + x_2 \geq 8$$

Carbohydrates Requirement:

$$x_1 + 2x_2 \geq 10$$

Fat Requirement:

$$x_1 + x_2 \geq 6$$

Non-negativity Constraints:

$$x_1 \geq 0, \quad x_2 \geq 0$$

Complete Linear Programming Model

$$
\begin{aligned}
\text{Minimize } Z &= 3x_1 + 2x_2 \\
\text{Subject to } 2x_1 + x_2 &\geq 8 \\
x_1 + 2x_2 &\geq 10 \\
x_1 + x_2 &\geq 6 \\
x_1 &\geq 0 \\
x_2 &\geq 0
\end{aligned}
$$

`Python` Implementation

```python
import pulp

# Create the optimization problem
model = pulp.LpProblem("Diet_Problem", pulp.LpMinimize)

# Define decision variables
x1 = pulp.LpVariable('Food_1', lowBound=0, cat='Continuous')
x2 = pulp.LpVariable('Food_2', lowBound=0, cat='Continuous')

# Define objective function (minimize cost)
model += 3*x1 + 2*x2, "Total_Cost"

# Add nutritional constraints
model += 2*x1 + x2 >= 8, "Protein_Requirement"
model += x1 + 2*x2 >= 10, "Carbohydrates_Requirement"
model += x1 + x2 >= 6, "Fat_Requirement"

# Solve the problem
model.solve()

# Display results
print("=== DIET PROBLEM OPTIMIZATION ===")
print(f"Solution Status: {pulp.LpStatus[model.status]}")
print(f"Minimum Daily Cost: ${pulp.value(model.objective):.2f}")
print(f"Optimal Food Quantities:")
print(f"  Food 1: {x1.varValue:.2f} kg")
print(f"  Food 2: {x2.varValue:.2f} kg")

# Verify nutritional intake
print(f"\nNutritional Analysis:")
print(f"Protein Intake: {2*x1.varValue + x2.varValue:.1f} units (Minimum: 8 units)")
print(f"Carbohydrates Intake: {x1.varValue + 2*x2.varValue:.1f} units (Minimum: 10 units)")
```

```
print(f"Fat Intake: {x1.varValue + x2.varValue:.1f} units (Minimum: 6 units)")

# Cost breakdown
print(f"\nCost Breakdown:")
print(f"Food 1 Cost: ${3*x1.varValue:.2f}")
print(f"Food 2 Cost: ${2*x2.varValue:.2f}")
print(f"Total Cost: ${pulp.value(model.objective):.2f}")
```

```
=== DIET PROBLEM OPTIMIZATION ===
Solution Status: Optimal
Minimum Daily Cost: $14.00
Optimal Food Quantities:
  Food 1: 2.00 kg
  Food 2: 4.00 kg

Nutritional Analysis:
Protein Intake: 8.0 units (Minimum: 8 units)
Carbohydrates Intake: 10.0 units (Minimum: 10 units)
Fat Intake: 6.0 units (Minimum: 6 units)

Cost Breakdown:
Food 1 Cost: $6.00
Food 2 Cost: $8.00
Total Cost: $14.00
```

2.10 Micro-Project 1: Campus City Emergency Supply Distribution

As an optimization analyst at **Campus City Logistics**, you've been tasked with designing the optimal supply distribution network for essential resources across campus facilities. The current ad-hoc system is inefficient and costly.

2.10.1 Problem Statement

Determine the optimal warehouse locations and distribution plan that minimizes total annual costs while meeting all facility demands, respecting warehouse capacity constraints, and operating within the allocated budget.

2.10.2 Facilities Data (From facilities.csv)

Based on our dataset, we have 15 facilities with the following key attributes:

Critical Facilities Selected for Micro-Project 1:

Facility ID	Facility Name	Type	Daily Demand
MED_CENTER	Campus Medical Center	Hospital	80 units
ENG_BUILDING	Engineering Building	Academic	30 units
SCIENCE_HALL	Science Hall	Academic	35 units
DORM_A	North Dormitory	Residential	55 units
DORM_B	South Dormitory	Residential	45 units
LIBRARY	Main Library	Academic	25 units

Total Daily Demand: 270 units

2.10.3 Warehouse Data (From warehouses.csv)

Warehouse ID	Warehouse Name	Daily Capacity	Construction Cost	Operational Cost/Day
WH_NORTH	North Campus Warehouse	400 units	$300,000	$800
WH_SOUTH	South Campus Warehouse	350 units	$280,000	$700
WH_EAST	East Gate Warehouse	450 units	$320,000	$900

Total Available Capacity: 1,200 units

2.10.4 Transportation Costs (From transportation_costs.csv)

- **Data Source**: Pre-calculated matrix with actual distances
- **Cost Range**: $3.68 - $5.03 per unit between selected locations
- **Calculation**: Based on real geographic coordinates using Haversine formula

2.10.5 Financial Constraints

- **Annual Budget**: $1,500,000
- **Operational Period**: 365 days (annual calculation)
- **Construction Cost**: Amortized over 10 years
- **All costs must be annualized**

2.10.6 Physical & Business Constraints

1. **Warehouse Selection**: Select exactly 2 warehouses for redundancy
2. **Demand Satisfaction**: Each facility must receive exactly its daily demand $\times$ 365
3. **Capacity Limits**: Shipments from each warehouse capacity $\times$ 365
4. **Budget Limit**: Total annual cost $1,500,000
5. **Non-negativity**: All shipment quantities 0

2.10.7 Learning Objectives

Technical Skills

- Formulate Mixed-Integer Linear Programming (MILP) problem
- Implement optimization model using PuLP
- Handle real geographic and cost data
- Validate constraint satisfaction

Analytical Skills

- Interpret optimization results in business context
- Analyze cost breakdown and efficiency metrics
- Provide actionable recommendations

Project Deliverables

1. Deliverable 1: Mathematical Formulation (3 Marks)
2. Deliverable 2: Report in .pdf format (7 Marks)

3 Module 2: Non-linear Programming with Python

3.1 Module Overview

3.1.1 Learning Objectives

- Understand the limitations of linear models and need for nonlinear optimization
- Master unconstrained optimization techniques (Golden Section, Fibonacci)
- Formulate and solve constrained nonlinear problems using KKT conditions
- Implement nonlinear optimization algorithms in Python
- Apply nonlinear methods to realistic cost modeling problems

3.1.2 Module Structure

- **Section 2.1**: Introduction to Nonlinear Optimization
- **Section 2.2**: Unconstrained Optimization Methods

- **Section 2.3**: Constrained Optimization & KKT Conditions
- **Section 2.4**: Python Implementation of Nonlinear Solvers
- **Section 2.5**: Advanced Applications & Micro-Project 2

3.1.3 Real-World Context

Extend the Campus City supply chain model with realistic nonlinear cost functions, environmental constraints, and multi-period optimization challenges.

3.2 Introduction to Nonlinear Optimization

Nonlinear programming (NLP) is an optimization technique used to find the best outcome (maximum profit, lowest cost, etc.) in a mathematical model where at least one of the objective functions or constraints is nonlinear . This means the relationship between variables cannot be represented on a straight line. NLP addresses the complexity found in real-world scenarios,

such as modeling physical systems or financial markets, where linear assumptions are often inadequate for accurate prediction and decision-making . Key components of an NLP problem include decision variables, an objective function to be optimized, and constraints that limit the possible solutions.

The primary purpose of using NLP is to provide more accurate and realistic models for optimization problems across various fields, including engineering design, economics, finance, and operations research. For example, in portfolio optimization, investors use NLP to maximize returns while balancing risk, which inherently involves nonlinear relationships between different assets. In chemical engineering, NLP helps optimize process yields and efficiency where reaction kinetics are nonlinear. It is applied when linear models fail to capture the true nature of the relationships between the inputs and outputs of a system.

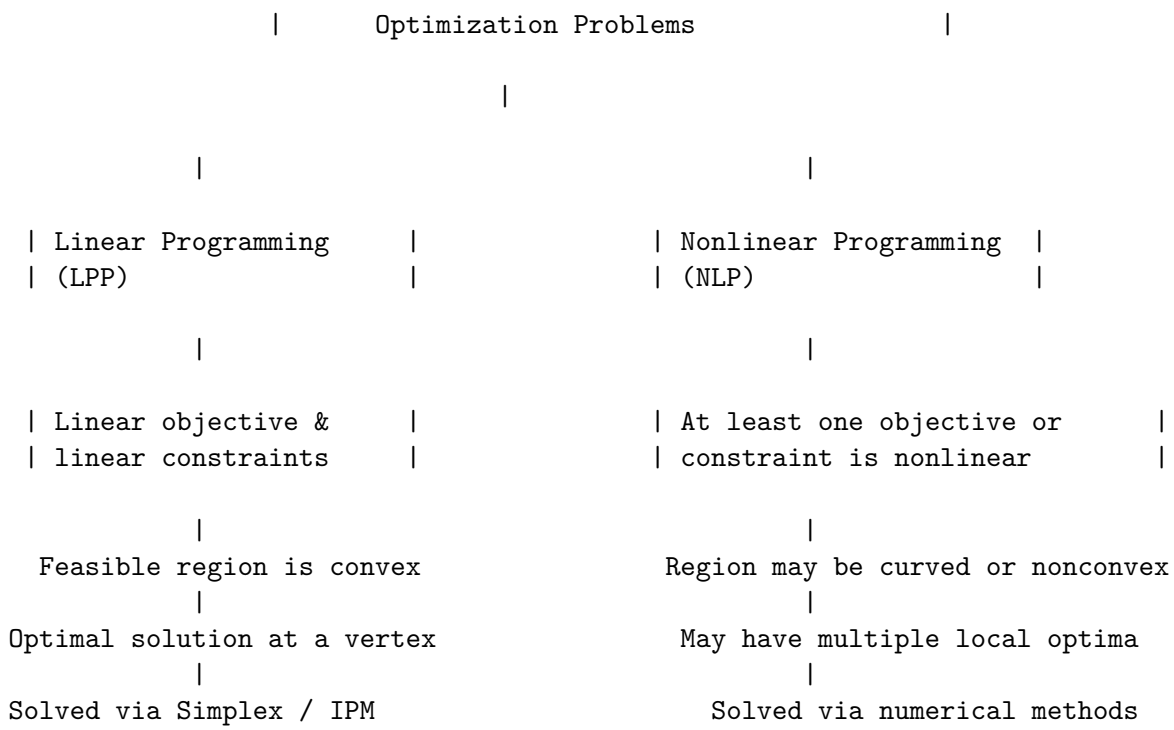
Solving NLP problems involves specialized algorithms, such as sequential quadratic programming (SQP), interior-point methods, or generalized reduced gradient methods. The "how much" refers to the computational resources and mathematical complexity required, which are often significantly greater than for linear programming. The complexity and effort vary widely based on the problem's size and structure (e.g., convexity). While the general approach is similar (define variables, objective, constraints), the execution demands sophisticated software and potentially substantial processing power to find the global optimum efficiently.

3.2.1 Theoretical Foundation

The fundamental theoretical difference between Linear Programming Problems (LPP) and NLP lies in the *mathematical nature of their objective functions and constraints*, which dictates the complexity of their feasible regions, solution methods, and optimality guarantees.

Feature	Linear Programming (LPP)	Nonlinear Programming (NLP)
Function Types	Objective function and all constraints must be linear.	At least one function (objective or a constraint) is nonlinear.
Feasible Region	The feasible region is a convex polyhedron (a shape with flat faces).	The feasible region can be non-convex, curved, or disconnected.
Optimality	A locally optimal solution is always a globally optimal solution. The optimum lies at a **corner point (vertex)** of the feasible region.	Solutions often only guarantee a local optimum. The global optimum is difficult to find without additional assumptions (e.g., convexity of the entire problem).

Feature	Linear Programming (LPP)	Nonlinear Programming (NLP)
Solution Methods	Solved primarily by the **Simplex Method** or **Interior Point Methods**, which systematically explore the corner points.	Requires more complex iterative numerical methods such as **Gradient Descent**, **Newton's Method**, or **Lagrangian heuristics**, often relying on derivatives.
Computational Complexity	Generally easier and faster to solve, even with a vast number of variables.	Generally more computationally intensive and difficult to solve, especially for large-scale, non-convex problems.



3.2.1.1 Why Nonlinear Optimization?

Linear programming assumes proportional relationships, but real-world problems often involve:

- **Economies of scale**: Decreasing marginal costs
- **Physical constraints**: Nonlinear relationships (distance vs fuel consumption)
- **Quality constraints**: Minimum/maximum thresholds with nonlinear effects
- **Environmental factors**: Quadratic cost functions for emissions

3.2.1.2 Mathematical Formulation

A general Nonlinear Programming (NLP) problem:

Objective Function:

$$\text{Minimize } f(\mathbf{x})$$

Subject to:

$$\begin{aligned} g_i(\mathbf{x}) &\leq 0, \quad i = 1, \ldots, m \\ h_j(\mathbf{x}) &= 0, \quad j = 1, \ldots, p \\ \mathbf{x} &\in \mathbb{R}^n \end{aligned}$$

Where:

- $f(\mathbf{x})$: Nonlinear objective function
- $g_i(\mathbf{x})$: Nonlinear inequality constraints
- $h_j(\mathbf{x})$: Nonlinear equality constraints

3.2.2 Sample Problem

Problem Statement: A delivery company has a cost function $C(d) = 500 + 2d + 0.1d^2$ where d is distance in km. Find the optimal delivery distance that minimizes cost per unit for a shipment of 100 units.

Mathematical Formulation:

$$\text{Minimize } \frac{C(d)}{100} = 5 + 0.02d + 0.001d^2$$

Solution:

1. Take derivative: $\frac{d}{dd}\left(5 + 0.02d + 0.001d^2\right) = 0.02 + 0.002d$
2. Set to zero: $0.02 + 0.002d = 0$
3. Solve: $d = -10$ km $\rightarrow$ Not feasible
4. Check boundaries: Minimum at $d = 0$ with cost \$5/unit

Interpretation: The cost function is convex and increasing, so minimum occurs at shortest distance.

3.2.3 Review Questions

1. What are three real-world scenarios where linear models fail and nonlinear optimization is required?
2. Explain the difference between convex and non-convex optimization problems. Why is this distinction important?
3. A manufacturing process has cost $C(x) = 1000 + 50x + 0.5x^2$ where x is production volume. Find the production level that minimizes average cost.
4. Why can't the Simplex method be directly applied to nonlinear optimization problems?
5. Describe a campus logistics scenario where nonlinear optimization would provide better results than linear programming.

NLP Vs LPP

The linear nature of LPP guarantees a single, well-defined feasible region and objective function behavior, allowing for robust algorithms like the Simplex method to efficiently find the absolute best (global) solution. NLP, by contrast, accommodates the complex, often non-convex, relationships found in many real-world problems. This flexibility comes at a theoretical and computational cost: the presence of curves and non-linear interactions means there may be multiple "local" optimal points, and standard algorithms may get stuck at one that is not the overall best solution. The theoretical foundation of NLP thus focuses heavily on iterative improvement, convergence properties, and finding efficient ways to navigate these more challenging landscapes.

3.3 Unconstrained Optimization Methods

Unconstrained Nonlinear Programming deals with optimizing a nonlinear objective function without any constraints on the decision variables. The objective is to determine a point $x^* \in \mathbb{R}^n$ such that:

$$\min_{x \in \mathbb{R}^n} f(x)$$

(or equivalently $\max f(x)$ depending on the problem context).

Intuition Behind Unconstrained NLP

The central idea in solving unconstrained NLP problems is to understand how the objective function surface behaves and to iteratively move toward regions where the function value improves.

Downhill Movement Intuition

For minimization problems, the objective is to move step-by-step in a direction that decreases the function value, much like descending a slope.

Hiking Analogy

Consider attempting to reach the lowest valley point in a mountainous landscape while blindfolded. The only available information is the slope beneath your feet. The safest strategy is always stepping in the direction that slopes downward most strongly.

Key points

- This *direction of steepest descent* is mathematically represented by the gradient $\nabla f(x)$.
- Repeated movement in this direction leads to progressively lower function values until a flat region is encountered. ## Link with Calculus

In single-variable calculus, stationary (optimal) points are found by solving:

$$\frac{df}{dx} = 0$$

In multi-variable calculus, this concept generalizes to:

$$\nabla f(x^*) = 0$$

A point where the gradient is zero is called a **critical point**. It may represent:

Type of Critical Point	Interpretation
Local Minimum	Valley bottom
Local Maximum	Hilltop
Saddle Point	A flat ridge point

To classify these points, second-order information is required.

First-Order Necessary Condition (FONC)

For a differentiable function $f(x)$, a point x^* can be a local minimum only if:

$$\nabla f(x^*) = 0$$

This guarantees flatness but **does not guarantee** an actual minimum.

Second-Order Sufficient Condition (SOSC)

Let $H(x)$ denote the **Hessian matrix** of second partial derivatives:

$$H(x) = \left[\frac{\partial^2 f}{\partial x_i \partial x_j}\right]$$

If x^* satisfies $\nabla f(x^*) = 0$ and

$$H(x^*) \text{ is positive definite}$$

then x^* is a **strict local minimum**.

Curvature Interpretation

Hessian Property	Interpretation
$H \succ 0$	Strict minimum (upward curvature)
$H \prec 0$	Maximum (downward curvature)
Indefinite	Saddle point (mixed curvature)

3.3.1 Algorithms for Unconstrained NLP

This module focuses specifically on *iteration-based direct search techniques*, which do not require gradient or Hessian calculations. Such algorithms are especially useful when:

- The derivative is difficult or impossible to compute
- The objective function is noisy or discontinuous
- Only function value evaluations are available

The algorithms covered in this module include:

3.3.1.1 Fibonacci Search Method

A bracketing method used for **one-dimensional** minimization.
This method iteratively narrows down an interval containing the minimum using Fibonacci numbers.

Key Concept

The interval size decreases proportionally according to Fibonacci sequence ratios, ensuring efficient narrowing.
This method is applicable to Smooth 1-D unimodal functions.

3.3.1.2 Golden Section Search Method

A more efficient improvement over Fibonacci search, eliminating the need to precompute steps. The method divides the interval using the *Golden Ratio*:

$$\phi = \frac{\sqrt{5}+1}{2} \approx 1.618$$

> **Key Concept**
>
> The interval is repeatedly reduced while maintaining internal division based on ϕ, ensuring optimal reduction at each iteration.

3.3.1.3 Hooke and Jeeves Pattern Search Method

A *direct search* method used for *multidimensional* optimization without derivatives.

> **Key Components**
>
> - **Exploratory search:** Tests moves along coordinate directions to find improvement.
> - **Pattern move:** Accelerates search in the identified promising direction.

3.3.1.4 Advantages

- Does not require derivative information
- Suitable for complex, non-smooth landscapes

Unconstrained NLP is guided by theoretical optimality conditions and solved computationally using iterative search techniques. In this module, emphasis is placed on **direct search algorithms** useful when derivatives are unavailable.

$$\nabla f(x^*) = 0 \quad \text{and} \quad H(x^*) \succ 0$$

The practical tools studied—**Fibonacci Search, Golden Section Search, and Hooke & Jeeves**—offer systematic methods for approaching optimal points when classical derivative-based methods cannot be applied.

3.3.1.5 Review Questions

1. Why does the condition $\nabla f(x^*) = 0$ not guarantee a minimum?
2. What role does curvature play in distinguishing minima from maxima?
3. Why are direct search methods such as Fibonacci or Golden Section useful?
4. Compare Hooke & Jeeves with gradient-based methods.
5. Explain why Golden Section is more efficient than Fibonacci Search.

3.3.2 Theoretical Foundation

3.3.2.1 One-Dimensional Search Methods

When dealing with single-variable nonlinear functions, we use specialized search techniques:

Golden Section Search: - Divide interval using golden ratio $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ - Progressively narrow search interval - Guaranteed convergence for unimodal functions

Fibonacci Search: - Uses Fibonacci sequence for interval reduction - Optimal in terms of function evaluations - Similar convergence properties to Golden Section

3.3.2.2 Mathematical Basis

For a unimodal function $f(x)$ on interval ([a,b]):

Golden Section Points:

$$x_1 = b - \frac{b-a}{\phi}$$
$$x_2 = a + \frac{b-a}{\phi}$$

Where $\phi = 1.618$

> **i** Alter:
>
> Instead of choosing $\phi = 1.618$, we can select the other Golder section ratio (< 1). In that case the section points can be calculated as:
>
> $$x_1 = b - \phi \cdot (b-a)$$
> $$x_2 = a + \phi \cdot (b-a)$$

Iteration: Compare $f(x_1)$ and $f(x_2)$, eliminate portion of interval based on which is larger.

3.3.3 Sample Problem 1

Problem Statement: Use Golden Section search to minimize $f(x) = (x-3)^2 + 2$ on interval [0, 5] with 4 iterations.

Solution: Iteration 1:

- $x_1 = 5 - \frac{5-0}{1.618} = 1.91$, $f(x_1) = 2.19$
- $x_2 = 0 + \frac{5-0}{1.618} = 3.09$, $f(x_2) = 2.01$
- Since $f(x_1) > f(x_2)$, new interval: [1.91, 5]

Iteration 2:

- $x_1 = 5 - \frac{5-1.91}{1.618} = 3.09$ (already computed)
- $x_2 = 1.91 + \frac{5-1.91}{1.618} = 3.82$, $f(x_2) = 2.67$
- Since $f(x_1) < f(x_2)$, new interval: [1.91, 3.82]

Continue for 4 iterations → Final interval: [2.85, 3.15] containing true minimum at x=3

3.3.4 Sample Problem 2

Problem Statement: A warehouse has energy consumption $E(w) = 1000 + 50w + 0.2w^2$ kWh, where w is weekly shipments (0 w 200). Find optimal shipment volume that minimizes energy per shipment.

Solution: Energy per shipment: $f(w) = \frac{1000}{w} + 50 + 0.2w$

Using Golden Section on [10, 200]:

- **Iteration 1:** Compare w=70.8 and w=139.2
- **Iteration 2:** Compare w=44.4 and w=70.8

- **Iteration 3:** Compare w=57.6 and w=70.8
- **Optimal:** w 70.7 shipments/week, energy/shipment 78.3 kWh

3.3.5 Sample problem 3

Minimize

$$f(x) = -\frac{1}{(x-1)^2}\left(\ln(x) - 2\left(\frac{x-1}{x+1}\right)\right)$$

such that x in $[1.5, 4.5]$ using the Golden Section Method.

3.3.6 Review Questions

1. Explain why Golden Section search is more efficient than exhaustive search for unimodal functions.
2. Compare Golden Section and Fibonacci search methods. When would you choose one over the other?
3. A transportation cost function is $C(d) = \frac{1000}{d} + 2d$ for d > 0. Use Golden Section to find the optimal distance in [10, 100] with 3 iterations.
4. What is a unimodal function and why is this property important for one-dimensional search methods?
5. How would you modify Golden Section search if you suspected multiple local minima in your search interval?

3.4 Constrained Optimization & KKT Conditions

3.4.1 Theoretical Foundation

3.4.1.1 Karush-Kuhn-Tucker (KKT) Conditions

For constrained nonlinear optimization, KKT conditions provide necessary conditions for optimality:

Primal Problem:

$$\begin{aligned} &\text{Minimize } f(\mathbf{x}) \\ &\text{Subject to } g_i(\mathbf{x}) \leq 0, \quad i = 1, \ldots, m \\ &\qquad\qquad h_j(\mathbf{x}) = 0, \quad j = 1, \ldots, p \end{aligned}$$

KKT Conditions:

1. **Stationarity:** $\nabla f(\mathbf{x}) + \sum \lambda_i \nabla g_i(\mathbf{x}) + \sum \mu_j \nabla h_j(\mathbf{x}) = 0$
2. **Primal Feasibility:** $g_i(\mathbf{x}) \leq 0, \quad h_j(\mathbf{x}) = 0$
3. **Dual Feasibility:** $\lambda_i \geq 0$
4. **Complementary Slackness:** $\lambda_i g_i(\mathbf{x}) = 0$

3.4.1.2 Interpretation

- **Lagrange Multipliers (,):** Sensitivity of objective to constraint changes
- **Complementary Slackness:** Inactive constraints have zero multipliers
- **Stationarity:** Balance between objective improvement and constraint satisfaction

3.4.2 Sample Problem

Problem Statement:

Minimize $f(x, y) = x^2 + y^2$ subject to $x + y \geq 4$.

Solution:

1. **Rewrite constraint:** $g(x, y) = 4 - x - y \leq 0$
2. **Lagrangian:** $L(x, y, \lambda) = x^2 + y^2 + \lambda(4 - x - y)$
3. **KKT Conditions:**
   - $\frac{\partial L}{\partial x} = 2x - \lambda = 0$
   - $\frac{\partial L}{\partial y} = 2y - \lambda = 0$
   - $\lambda(4 - x - y) = 0$
   - $4 - x - y \leq 0$, \$ 0 \$
4. **Case 1:** $= 0 \rightarrow$ x=0, y=0 $\rightarrow$ violates constraint
5. **Case 2:** 4 - x - y = 0 $\rightarrow$ from stationarity: x=y= /2 $\rightarrow$ 4 - /2 - /2 = 0 $\rightarrow$ =4
6. **Solution:** x=2, y=2, =4, f=8

3.4.3 Sample Problem

Problem Statement: Campus City wants to minimize transportation cost $C(d) = 2d + 0.1d^2$ while ensuring at least 80% of routes are under 3km. Formulate as KKT problem.

Solution:

Mathematical Formulation:

- Objective: Minimize $\sum 2d_i + 0.1d_i^2$
- Constraint: $\frac{\text{count}(d_i \leq 3)}{n} \geq 0.8$

KKT Interpretation:

- Lagrange multiplier indicates cost increase for relaxing the 80% constraint
- Helps decide if expanding the constraint is cost-effective

3.4.4 Review Questions

1. Explain the economic interpretation of Lagrange multipliers in constrained optimization.
2. What does complementary slackness tell us about active vs inactive constraints at the optimal solution?
3. Solve using KKT: Minimize $f(x) = x^2$ subject to $x \geq 2$.

4. How do KKT conditions extend the method of Lagrange multipliers for inequality constraints?
5. In a campus logistics context, what would a high Lagrange multiplier for an environmental constraint indicate?

3.5 Python Implementation of Nonlinear Solvers

3.5.1 Theoretical Foundation

3.5.1.1 Python Optimization Ecosystem

- **scipy.optimize**: Comprehensive optimization toolkit
- **minimize() function**: Unified interface for multiple algorithms
- **Algorithm choices**: SLSQP, trust-constr, COBYLA for constrained problems

3.5.1.2 Key Functions

```
from scipy.optimize import minimize

# For unconstrained problems
result = minimize(fun, x0, method='BFGS')

# For constrained problems
result = minimize(fun, x0, constraints=constraints, method='SLSQP')
```

> **i** Practical Considerations
>
> - Initial guess (x_0): Critical for convergence
> - Gradient provision: Can significantly improve performance
> - Constraint formulation: Proper bounds and constraint objects

3.5.2 Sample Problem

Problem Statement: Implement Golden Section search in `Python` for $f(x) = (x-4)^2 + 3$ on $[0, 10]$.

Solution:

```python
import math

def golden_section_search(f, a, b, tol=1e-6, max_iter=100):
    phi = (1 + math.sqrt(5)) / 2
    for i in range(max_iter):
        x1 = b - (b - a) / phi
        x2 = a + (b - a) / phi

        if f(x1) > f(x2):
            a = x1
        else:
            b = x2

        if abs(b - a) < tol:
            return (a + b) / 2
    return (a + b) / 2

# Test the function
def objective(x):
    return (x - 4)**2 + 3

result = golden_section_search(objective, 0, 10)
print(f"Optimal x: {result:.4f}")  # Output: Optimal x: 4.0000
```

```
Optimal x: 4.0000
```

3.5.3 Solution of constrained NLPP

To solve NLPP using KKT method, we will use `solve()` function from the `sympy` library. Key functions required for solution will be imported

```python
from sympy import symbols, diff, solve
```

> **Example 1:** Minimize $f(x, y) = x^2 + y^2$ subject to $x + y \geq 4$ (Standard form: $4 - x - y \leq 0$).

`SymPy` does not inherently "solve" inequality constraints in the same way a numerical solver (like `SciPy`) does. Instead, you must manually transform the inequality into an equation using the Complementary Slackness condition from the KKT framework.Here is the logic: An inequality constraint $g(x) \leq 0$ is handled by introducing a Lagrange multiplier λ and adding the equation $\lambda \cdot g(x) = 0$ to your system.

The 3-Step "Filter" WorkflowSince SymPy's solve function returns all mathematical solutions (stationary points), you must add a "Filter" step to check the inequalities manually. - *Formulate:* Convert inequalities to KKT equations (Stationarity + Complementary Slackness). - *Solve:* Use `solve()` to find all candidate points. - *Filter:* Discard solutions that violate Primal Feasibility ($g(x) \leq 0$) or Dual Feasibility ($\lambda \geq 0$).

Python code:

```
from sympy import symbols, diff, solve

# 1. Define Variables
x, y = symbols('x y', real=True)
lam = symbols('lambda', real=True) # Lagrange Multiplier

# 2. Define Functions
f = x**2 + y**2           # Objective
g = 4 - x - y             # Inequality constraint (g <= 0)

# 3. Formulate Lagrangian
# L = f + lambda * g
L = f + lam * g

# 4. Generate KKT Equations
# A. Stationarity: Gradient of L must be zero
eq1 = diff(L, x)
eq2 = diff(L, y)

# B. Complementary Slackness: lambda * constraint = 0
eq3 = lam * g

# 5. Solve the System of Equations
# We ask SymPy to find values for x, y, and lambda that satisfy these 3 equalities
solutions = solve([eq1, eq2, eq3], (x, y, lam), dict=True)

print(f"Candidate Solutions found: {len(solutions)}")

# 6. Filter Solutions (The Crucial Step for Inequalities)
print("\n--- Validation Step ---")
valid_solutions = []

for sol in solutions:
    x_val = sol[x]
    y_val = sol[y]
```

```
    lam_val = sol[lam]

    # Check Primal Feasibility: g(x) <= 0
    # (Using a small tolerance for floating point comparisons if needed)
    primal_check = (4 - x_val - y_val) <= 0

    # Check Dual Feasibility: lambda >= 0
    dual_check = lam_val >= 0

    print(f"Testing Point ({x_val}, {y_val}) with lambda={lam_val}:")
    print(f"  Constraint g <= 0? {primal_check}")
    print(f"  Dual lambda >= 0? {dual_check}")

    if primal_check and dual_check:
        print(" VALID Optimal Solution")
        valid_solutions.append(sol)
    else:
        print(" INVALID (Violates KKT conditions)")

# Output Final Result
if valid_solutions:
    opt = valid_solutions[0]
    print(f"\nOptimal Solution: x={opt[x]}, y={opt[y]}, cost={opt[x]**2 + opt[y]**2}")
```

```
Candidate Solutions found: 2

--- Validation Step ---
Testing Point (0, 0) with lambda=0:
  Constraint g <= 0? False
  Dual lambda >= 0? True
 INVALID (Violates KKT conditions)
Testing Point (2, 2) with lambda=4:
  Constraint g <= 0? True
  Dual lambda >= 0? True
 VALID Optimal Solution

Optimal Solution: x=2, y=2, cost=8
```

Example 2: Solve $f(x_1, x_2) = -x_1^2 - x_2^2$; $-x_1 - x_2 \leq -1$; $-2x_1 + 2x_2 \leq 1$; $x_1, x_2 \geq 0$.

```
import sympy as sp

# 1. Define Variables
x1, x2 = sp.symbols('x1 x2', real=True)
lam1, lam2 = sp.symbols('lambda1 lambda2', real=True)

# 2. Define Objective (Maximization)
f = -x1**2 - x2**2

# 3. Define Constraints in Standard Form (g(x) <= 0)
# Original: -x1 - x2 >= -1  -> x1 + x2 <= 1 -> x1 + x2 - 1 <= 0
g1 = x1 + x2 - 1

# Original: -2x1 + 2x2 <= 1 -> -2x1 + 2x2 - 1 <= 0
g2 = -2*x1 + 2*x2 - 1

# 4. Formulate Lagrangian for Maximization (L = f - lambda*g)
L = f - lam1 * g1 - lam2 * g2

# 5. KKT Equations
# A. Stationarity (Gradient of L w.r.t x must be 0)
stat_x1 = sp.diff(L, x1)
stat_x2 = sp.diff(L, x2)

# B. Complementary Slackness (lambda * g = 0)
slack_1 = lam1 * g1
slack_2 = lam2 * g2

# 6. Solve the System
print("Solving KKT system...")
# We assume non-negativity of x (x >= 0) is a boundary check rather than an explicit Lagrang
# to keep the symbolic solver efficient for this demo.
solutions = sp.solve([stat_x1, stat_x2, slack_1, slack_2], (x1, x2, lam1, lam2), dict=True)

# 7. Validation & Filtering
print(f"\nFound {len(solutions)} candidate points. Filtering for optimality...")
print("-" * 75)
print(f"{'Point (x1, x2)':<20} | {'Multipliers':<20} | {'Objective':<10} | {'Status'}")
print("-" * 75)

best_f = -float('inf')
best_sol = None
```

```python
for sol in solutions:
    # Extract numerical values
    x1_v = float(sol[x1])
    x2_v = float(sol[x2])
    l1_v = float(sol[lam1])
    l2_v = float(sol[lam2])

    status = "Valid"

    # Check 1: Non-negativity of variables (x >= 0)
    if x1_v < -1e-7 or x2_v < -1e-7:
        status = "Invalid (x < 0)"

    # Check 2: Primal Feasibility (g <= 0)
    elif (x1_v + x2_v - 1) > 1e-7:
        status = "Invalid (g1 > 0)"
    elif (-2*x1_v + 2*x2_v - 1) > 1e-7:
        status = "Invalid (g2 > 0)"

    # Check 3: Dual Feasibility (lambda >= 0 for Max problem with L = f - lg)
    elif l1_v < -1e-7 or l2_v < -1e-7:
        status = "Invalid (lambda < 0)"

    # Calculate Objective
    obj_val = -x1_v**2 - x2_v**2

    print(f"({x1_v:.4f}, {x2_v:.4f})   | L1={l1_v:.4f}, L2={l2_v:.4f} | {obj_val:.4f}     | 

    if "Valid" in status and obj_val > best_f:
        best_f = obj_val
        best_sol = (x1_v, x2_v)

print("-" * 75)
if best_sol:
    print(f"\n Optimal Solution: x1 = {best_sol[0]}, x2 = {best_sol[1]}")
    print(f"   Max Objective Value = {best_f}")
```

```
Solving KKT system...

Found 4 candidate points. Filtering for optimality...
---------------------------------------------------------------------------
```

```
Point (x1, x2)       | Multipliers          | Objective  | Status
-------------------------------------------------------------------------
(0.2500, 0.7500)   | L1=-1.0000, L2=-0.2500 | -0.6250     | Invalid (lambda < 0)
(0.5000, 0.5000)   | L1=-1.0000, L2=0.0000 | -0.5000     | Invalid (lambda < 0)
(-0.2500, 0.2500)   | L1=0.0000, L2=-0.2500 | -0.1250     | Invalid (x < 0)
(0.0000, 0.0000)   | L1=0.0000, L2=0.0000 | -0.0000     | Valid
-------------------------------------------------------------------------

 Optimal Solution: x1 = 0.0, x2 = 0.0
   Max Objective Value = -0.0
```

Example 3: Minimize $f(x_1, x_2) = 2x_1 + 6x_2$ subject to $x_1 - x_2 \leq 0$; $x_1^2 + x_2^2 \geq 4$; $x_1, x_2 \geq 0$.

Mathematical Formulation for `Sympy` requires constraints to be in the form $g(x) \geq 0$. We must rearrange the given inequalities: - *Objective:* Minimize $f(x) = 2x_1 + 6x_2$ - *Constraint 1:* $x_1 - x_2 \leq 0 \implies x_2 - x_1 \geq 0$ - *Constraint 2:* $x_1^2 + x_2^2 \geq 4 \implies x_1^2 + x_2^2 - 4 \geq 0$ - *Bounds:* $x_1, x_2 \geq 0$

```python
import sympy as sp

# 1. Define Symbols
x1, x2 = sp.symbols('x1 x2', real=True)
lam1, lam2 = sp.symbols('lambda1 lambda2', real=True)

# 2. Define Objective (Minimization)
f = 2*x1 + 6*x2

# 3. Define Constraints in Standard Form (g(x) <= 0)
# Constraint 1: x1 - x2 <= 0
g1 = x1 - x2

# Constraint 2: x1^2 + x2^2 >= 4  --> 4 - x1^2 - x2^2 <= 0
g2 = 4 - x1**2 - x2**2

# 4. Formulate Lagrangian (Minimization: L = f + sum(lambda * g))
L = f + lam1 * g1 + lam2 * g2

# 5. KKT Equations
# A. Stationarity (Gradient of L = 0)
eq_x1 = sp.diff(L, x1)
eq_x2 = sp.diff(L, x2)
```

```
# B. Complementary Slackness (lambda * g = 0)
eq_slack1 = lam1 * g1
eq_slack2 = lam2 * g2

# 6. Solve the System
print("Solving KKT system equations...")
# This solves for stationary points and intersection points
solutions = sp.solve([eq_x1, eq_x2, eq_slack1, eq_slack2], (x1, x2, lam1, lam2), dict=True)

# 7. Filter & Validate Solutions
print(f"\nFound {len(solutions)} candidate points. Validating KKT conditions...")
print("-" * 85)
print(f"{'Point (x1, x2)':<20} | {'Multipliers':<20} | {'Cost':<10} | {'Status'}")
print("-" * 85)

optimal_val = float('inf')
optimal_sol = None

for sol in solutions:
    # Extract values (convert to float for comparison)
    x1_v = float(sol[x1])
    x2_v = float(sol[x2])
    l1_v = float(sol[lam1])
    l2_v = float(sol[lam2])

    # Validation Logic
    status = "Valid"

    # 1. Domain Check (x >= 0)
    if x1_v < -1e-7 or x2_v < -1e-7:
        status = "Invalid (x < 0)"

    # 2. Primal Feasibility (g <= 0)
    # Check g1: x1 - x2 <= 0
    elif (x1_v - x2_v) > 1e-7:
        status = "Invalid (g1 > 0)"
    # Check g2: 4 - x1^2 - x2^2 <= 0
    elif (4 - x1_v**2 - x2_v**2) > 1e-7:
        status = "Invalid (g2 > 0)"

    # 3. Dual Feasibility (lambda >= 0 for Minimization)
    elif l1_v < -1e-7 or l2_v < -1e-7:
```

```
        status = "Invalid (lambda < 0)"

    # Calculate Cost
    cost = 2*x1_v + 6*x2_v

    print(f"({x1_v:.4f}, {x2_v:.4f})   | L1={l1_v:.4f}, L2={l2_v:.4f} | {cost:.4f}    | {sta

    if "Valid" in status and cost < optimal_val:
        optimal_val = cost
        optimal_sol = (x1_v, x2_v)

print("-" * 85)
if optimal_sol:
    print(f"\n Optimal Solution: x1 = {optimal_sol[0]:.4f}, x2 = {optimal_sol[1]:.4f}")
    print(f"   Minimum Value = {optimal_val:.4f}")

    # Insight for the user
    print("\nInsight: The solution lies at the intersection of the line x1=x2 and the circle
```

```
Solving KKT system equations...

Found 4 candidate points. Validating KKT conditions...
-------------------------------------------------------------------------------------
Point (x1, x2)       | Multipliers          | Cost       | Status
-------------------------------------------------------------------------------------
(-0.6325, -1.8974)   | L1=0.0000, L2=-1.5811 | -12.6491    | Invalid (x < 0)
(0.6325, 1.8974)   | L1=0.0000, L2=1.5811 | 12.6491    | Valid
(-1.4142, -1.4142)   | L1=2.0000, L2=-1.4142 | -11.3137    | Invalid (x < 0)
(1.4142, 1.4142)   | L1=2.0000, L2=1.4142 | 11.3137    | Valid
-------------------------------------------------------------------------------------

 Optimal Solution: x1 = 1.4142, x2 = 1.4142
   Minimum Value = 11.3137

Insight: The solution lies at the intersection of the line x1=x2 and the circle.
```

3.5.4 Support Vector Machine (SVM) Theory: A Constrained Optimization Perspective

Introduction & Geometric Intuition

The Support Vector Machine (SVM) is a supervised learning algorithm that finds the optimal hyperplane to separate two classes of data. Unlike other linear classifiers, SVM seeks the **widest possible separation** (margin) between the classes.

The Hyperplane Equation

A hyperplane is defined by the equation:

$$w^T x + b = 0$$

- w: The weight vector (normal to the hyperplane). - b: The bias term (position relative to origin). - x: The feature vector of a data point.

The Concept of the Margin

The "Margin" is the physical distance between the decision boundary and the closest data points (Support Vectors).

- **Positive Boundary:** $w^T x + b = +1$
- **Negative Boundary:** $w^T x + b = -1$

Mathematical Derivation of the Margin

To calculate the width of the margin (M), consider two support vectors, x_+ on the positive boundary and x_- on the negative boundary.

1. **Equations:**
   - $w^T x_+ + b = 1$
   - $w^T x_- + b = -1$
2. **Difference:** Subtracting the second from the first:
$$w^T (x_+ - x_-) = 2$$
3. **Projection:** The physical distance M is the projection of $(x_+ - x_-)$ onto the unit normal vector $\frac{w}{||w||}$:
$$M = (x_+ - x_-) \cdot \frac{w}{||w||} = \frac{w^T (x_+ - x_-)}{||w||} = \frac{2}{||w||}$$

Key Insight: To **maximize** the margin ($M = \frac{2}{||w||}$), we must **minimize** the norm of the weight vector ($||w||$).

The Constraint: Sign Agreement

For a classifier to be valid, it must correctly classify data points *outside* the margin. * If $y_i = +1$, we require $w^T x_i + b \geq 1$. * If $y_i = -1$, we require $w^T x_i + b \leq -1$.

These two conditions can be compressed into a single inequality:

$$y_i(w^T x_i + b) \geq 1 \quad \forall i$$

This constraint ensures that every point is both correctly classified and located safely outside the "no-man's land" of the margin.

The Primal Optimization Problem

Minimizing $||w||$ involves a square root, which is computationally difficult. Instead, we minimize $\frac{1}{2}||w||^2$, which is a convex quadratic function with the same optimum.

Standard Form (Hard Margin SVM):

$$\begin{aligned} \text{Minimize} \quad & f(w,b) = \frac{1}{2}||w||^2 \\ \text{Subject to} \quad & y_i(w^T x_i + b) \geq 1, \quad i = 1, \ldots, N \end{aligned}$$

Solving via Lagrange Multipliers (KKT Conditions) To solve this constrained problem, we construct the *Lagrangian Function*:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}||w||^2 - \sum_{i=1}^{N} \alpha_i [y_i(w^T x_i + b) - 1]$$

where $\alpha_i \geq 0$ are the Lagrange Multipliers.

The Karush-Kuhn-Tucker (KKT) Conditions

For optimality, the following conditions must hold:

1. **Stationarity ($\nabla \mathcal{L} = 0$):**
   - $\nabla_w \mathcal{L} = w - \sum \alpha_i y_i x_i = 0 \implies \mathbf{w = \sum\ _i y_i x_i}$
   - $\nabla_b \mathcal{L} = -\sum \alpha_i y_i = 0 \implies \mathbf{\sum\ _i y_i = 0}$
2. **Primal Feasibility:**
   - $y_i(w^T x_i + b) - 1 \geq 0$
3. **Dual Feasibility:**
   - $\alpha_i \geq 0$
4. **Complementary Slackness:**
   - $\alpha_i [y_i(w^T x_i + b) - 1] = 0$

- *Interpretation:* If $\alpha_i > 0$, the point lies exactly on the margin ($y_i(w^T x + b) = 1$). These points are the **Support Vectors**. If $\alpha_i = 0$, the point is irrelevant to the boundary.

Example Calculation

Data: Class +1 at $(2, 2)$, Class -1 at $(0, 0)$.

1. **Lagrangian Formulation:**

$$\mathcal{L} = \frac{1}{2}(w_1^2 + w_2^2) - \alpha_1(2w_1 + 2w_2 + b - 1) - \alpha_2(-b - 1)$$

2. **Solving:**
   - From stationarity: $w_1 = 2\alpha, w_2 = 2\alpha$.
   - From sum constraint: $\alpha_1 = \alpha_2 = \alpha$.
   - From active constraints: $b = -1$, $w_1 + w_2 = 1$.
3. **Result:**
   - $\alpha = 0.25$
   - **Optimal Weights:** $w = [0.5, 0.5]$
   - **Optimal Bias:** $b = -1$
   - **Margin Width:** $2/||w|| \approx 2.82$

Python solution

```
import sympy as sp

# 1. Define Variables
w1, w2, b = sp.symbols('w1 w2 b', real=True)
lam1, lam2 = sp.symbols('lambda1 lambda2', real=True)

# 2. Define Objective (Minimize 1/2 ||w||^2)
f = 0.5 * (w1**2 + w2**2)

# 3. Define Constraints (g <= 0)
# Constraint for Point A (2, 2): 1 - y(wx+b) <= 0
g1 = 1 - (1 * (2*w1 + 2*w2 + b))
# Constraint for Point B (0, 0): 1 - y(wx+b) <= 0
g2 = 1 - (-1 * (0*w1 + 0*w2 + b))

# 4. Formulate Lagrangian
L = f + lam1 * g1 + lam2 * g2
```

```
# 5. KKT Conditions
# A. Stationarity (Gradient of L = 0)
stat_w1 = sp.diff(L, w1)
stat_w2 = sp.diff(L, w2)
stat_b  = sp.diff(L, b)

# B. Complementary Slackness (lambda * g = 0)
slack_1 = lam1 * g1
slack_2 = lam2 * g2

# 6. Solve the System
print("Solving KKT system...")
# We explicitly ask for a dictionary result
solutions = sp.solve([stat_w1, stat_w2, stat_b, slack_1, slack_2],
                     (w1, w2, b, lam1, lam2), dict=True)

# 7. Validate Solutions
print(f"\nFound {len(solutions)} candidate solutions. Validating...")
print("-" * 60)

for sol in solutions:
    # Use .get() to handle cases where a variable is free/undetermined
    # If b is not in solution, it usually implies it's arbitrary in that specific branch
    # (e.g. if w=0, lambda=0, b might not be constrained by stationarity).
    # We default to 0.0 for checking purposes or flag it.
    try:
        val_w1 = float(sol.get(w1, 0))
        val_w2 = float(sol.get(w2, 0))
        val_b  = float(sol.get(b, 0))
        val_l1 = float(sol.get(lam1, 0))
        val_l2 = float(sol.get(lam2, 0))
    except TypeError:
        print("Skipping a symbolic/complex solution that couldn't be converted to float.")
        continue

    status = "Valid Optimal Solution"

    # Check Primal Feasibility (g <= 0)
    # Using a small tolerance (1e-6)
    if (1 - 2*val_w1 - 2*val_w2 - val_b) > 1e-6:
        status = "Invalid (Violates Point A Constraint)"
    elif (1 + val_b) > 1e-6:
```

```
        status = "Invalid (Violates Point B Constraint)"

    # Check Dual Feasibility (lambda >= 0)
    elif val_l1 < -1e-6 or val_l2 < -1e-6:
        status = "Invalid (Negative Multiplier)"

    # Print results
    print(f"Solution: w=({val_w1:.4f}, {val_w2:.4f}), b={val_b:.4f}")
    print(f"Multipliers:  1={val_l1:.4f},  2={val_l2:.4f}")
    print(f"Status: {status}")
    print("-" * 60)
```

```
Solving KKT system...

Found 3 candidate solutions. Validating...
------------------------------------------------------------
Solution: w=(0.0000, 0.0000), b=0.0000
Multipliers:  1=0.0000,  2=0.0000
Status: Invalid (Violates Point A Constraint)
------------------------------------------------------------
Solution: w=(0.0000, 0.0000), b=-1.0000
Multipliers:  1=0.0000,  2=0.0000
Status: Invalid (Violates Point A Constraint)
------------------------------------------------------------
Solution: w=(0.5000, 0.5000), b=-1.0000
Multipliers:  1=0.2500,  2=0.2500
Status: Valid Optimal Solution
------------------------------------------------------------
```

3.5.5 Principal Component Analysis (PCA)

While often viewed as linear algebra, PCA is fundamentally a constrained optimization problem seeking the direction of maximum variance.

The Formulation

- **Objective:** Find a direction vector w that maximizes the variance of the projected data.

$$\text{Maximize } f(w) = w^T \Sigma w$$

(where Σ is the Covariance Matrix)

- **Constraint:** To prevent the vector length from growing infinitely to cheat the maximization, we fix its length to 1.

$$\text{Subject to } g(w) = w^T w - 1 = 0$$

The Lagrangian Solution

We formulate the Lagrangian with multiplier λ:

$$\mathcal{L}(w, \lambda) = w^T \Sigma w - \lambda(w^T w - 1)$$

- **Finding the Optimum:** Differentiating with respect to w yields:

$$2\Sigma w - 2\lambda w = 0 \implies \Sigma w = \lambda w$$

> **Insight**
>
> The optimal direction w is simply the **Eigenvector** of the covariance matrix, and the Lagrange multiplier λ is the **Eigenvalue** (representing the amount of variance).

3.5.6 Ridge Regression (L2 Regularization)

Ridge regression modifies standard least-squares fitting by imposing a budget on the size of the coefficients to prevent overfitting.

The Formulation

- **Objective:** Minimize the sum of squared prediction errors.

$$\text{Minimize } ||y - X\beta||^2$$

- **Constraint:** The total size (squared magnitude) of the coefficients must not exceed a budget t.

$$\text{Subject to } ||\beta||^2 \leq t$$

The Lagrangian Solution

$$\mathcal{L}(\beta, \lambda) = ||y - X\beta||^2 + \lambda(||\beta||^2 - t)$$

- **Resulting Update Rule:** Solving for $\nabla \mathcal{L} = 0$ gives the famous closed-form solution:

$$\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$$

Insight:

The regularization parameter λ used in code is actually the Lagrange Multiplier corresponding to the weight constraint.

3.5.7 Maximum Entropy Models (MaxEnt)

Used in Natural Language Processing and ecology, `MaxEnt` seeks the most "unbiased" probability distribution that still fits the observed data.

The Formulation

- **Objective:** Maximize the Entropy (uncertainty) of the distribution $p(x)$.

$$\text{Maximize } H(p) = -\sum p(x) \log p(x)$$

- **Constraints:**

1. **Normalization:** The probabilities must sum to 1.

$$\sum p(x) = 1$$

2. **Feature Matching:** The expected value of features in our model must match the empirical averages seen in the training data.

$$\sum p(x) f_i(x) = \mathbb{E}_{data}[f_i]$$

The Lagrangian Solution

Solving this Lagrangian system yields the **Gibbs (Softmax) Distribution**:

$$p(y|x) = \frac{1}{Z} \exp\left(\sum \lambda_i f_i(x, y)\right)$$

Insights:

This derivation serves as the theoretical foundation for **Logistic Regression** and the final layer of most **Neural Networks**.

4 Module 3: Project Planning and Optimization Techniques

4.1 Module Overview

4.1.1 Learning Objectives

- Master project planning methodologies: CPM and PERT
- Implement network analysis for project scheduling
- Create and interpret Gantt charts for project visualization
- Apply heuristic algorithms for optimization problems
- Develop practical scheduling solutions for campus projects

4.1.2 Module Structure

- **Section 3.1**: Introduction to Project Planning
- **Section 3.2**: Critical Path Method (CPM)
- **Section 3.3**: Program Evaluation and Review Technique (PERT)
- **Section 3.4**: Gantt Charts and Project Visualization
- **Section 3.5**: Heuristic Algorithms and Local Search
- **Section 3.6**: Micro-Project 3: Campus Construction Planning

4.1.3 Real-World Context

In Module 1 and 2, we looked at how to optimize a system when we know the variables and the constraints. But in the real world—especially in engineering and software development—**time** is the most precious constraint of all.

Have you ever wondered how massive projects, like building our new campus library or launching a software product like ChatGPT, actually finish on time? It isn't magic; it's mathematics.

In this module, we are going to learn the art and science of **Scheduling**. We will move from "I hope we finish on time" to "I calculate we have a 95% probability of finishing by Friday." Apply

project planning techniques to campus construction projects, facility maintenance schedules, and resource allocation for campus events.

4.2 Introduction to Project Planning

4.2.1 The Engineering of Time

Project planning is essentially an optimization problem where we try to minimize time (T) or Cost (C) while respecting a complex web of dependencies.

Imagine you are cooking a meal. You cannot fry the onions before you chop them. That is a **dependency**. However, you *can* boil the water while you chop the onions. That is **parallelism**. Project planning is simply the mathematical modeling of these dependencies and parallel tasks.

4.2.2 Theoretical Foundation

We represent a project as a **Directed Acyclic Graph (DAG)**, denoted as $G = (V, E)$.

- **Vertices (V):** These are events or milestones (e.g., "Foundation poured").
- **Edges (E):** These are the activities that take time (e.g., "Pouring Concrete - 3 days").
- **Weights:** The duration or cost attached to an edge.

4.2.2.1 Mathematical Representation

A project can be represented as a directed graph $G = (V, E)$ where:

- **Vertices (V)**: Represent project milestones or activity completion points
- **Edges (E)**: Represent activities with associated durations
- **Weights**: Activity durations or costs

> **i** Insight
>
> Why "Acyclic"? Because if you have a cycle (A depends on B, B depends on A), you have an infinite loop. In project management, that means the project never finishes! We must avoid cycles.

4.2.3 Sample Scenario: The Library Construction

Let's look at a concrete example. We are building a library. Here is the logic:

Activity	Description	Predecessors	Duration (weeks)
A	Site Prep	-	2
B	Foundation	A	4
C	Framing	B	6
D	Electrical	C	3
E	Plumbing	C	3
F	Interior	D, E	4
G	Exterior	C	2
H	Inspection	F, G	1

Do you see activity **F**? It cannot start until **both** D and E are finished. This is where the math gets interesting.

4.2.3.1 What is Project Planning?

Project planning involves defining project objectives, determining tasks and resources needed, and establishing timelines for project completion. In optimization context, it's about finding the most efficient sequence of activities to minimize time or cost.

4.2.3.2 Key Components of Project Planning

1. **Activities/Tasks**: Discrete work elements with defined durations
2. **Dependencies**: Precedence relationships between activities
3. **Resources**: Personnel, equipment, and materials required
4. **Time Estimates**: Duration for each activity
5. **Milestones**: Significant checkpoints in project timeline

4.2.3.3 Project Management Objectives

1. **Time Minimization**: Complete project in shortest possible time
2. **Cost Optimization**: Minimize total project cost
3. **Resource Leveling**: Smooth resource utilization over time
4. **Risk Mitigation**: Identify and manage critical activities

4.2.3.4 Campus City Applications

- Construction of new campus facilities
- Maintenance scheduling for existing buildings
- Event planning and resource allocation
- Academic calendar optimization
- Emergency response planning

4.2.4 Sample Problem

Problem Statement: Campus City is planning a new library construction with the following activities:

Activity	Description	Immediate Predecessors	Duration (weeks)
A	Site preparation	-	2
B	Foundation work	A	4
C	Structural framing	B	6
D	Electrical work	C	3
E	Plumbing work	C	3
F	Interior work	D, E	4
G	Exterior finishing	C	2
H	Final inspection	F, G	1

Construct the project network and identify the project completion time.

Solution Approach: We'll represent this as a network diagram with nodes as activity completion points and edges as activities. The solution will be developed in Section 3.2.

4.2.5 Review Questions

1. What are the key differences between project planning and continuous optimization problems?
2. Explain how project dependencies create sequencing constraints that must be respected in scheduling.
3. Describe three campus scenarios where project planning techniques would provide significant benefits over ad-hoc scheduling.
4. What information is minimally required to begin project planning for a campus construction project?
5. How does uncertainty in activity durations affect project planning decisions?

4.3 Critical Path Method (CPM)

4.3.1 The "Bottleneck" Concept

The Critical Path Method (CPM) is one of the most powerful tools in an engineer's toolkit. Here is the intuition: **A project is only as fast as its slowest sequence of dependent tasks.**

We call this longest sequence the **Critical Path**. If any activity on this path is delayed by one day, the *entire project* is delayed by one day.

4.3.1.1 What is Critical Path Method?

CPM is an algorithm for scheduling project activities that:

- Identifies critical activities that cannot be delayed without delaying the project
- Calculates the minimum project duration
- Determines float/slack time for non-critical activities

4.3.1.2 CPM Terminology

1. **Earliest Start Time (ES)**: Earliest time an activity can begin
2. **Earliest Finish Time (EF)**: ES + Activity Duration
3. **Latest Start Time (LS)**: Latest time an activity can begin without delaying project
4. **Latest Finish Time (LF)**: LS + Activity Duration
5. **Float/Slack**: LF - EF or LS - ES (flexibility in scheduling)
6. **Critical Path**: Sequence of activities with zero float

4.3.1.3 CPM Algorithm Steps

Forward Pass (Calculate ES, EF):

1. Start with initial activity: ES = 0, EF = Duration
2. For each subsequent activity: ES = max(EF of all predecessors)
3. EF = ES + Duration
4. Project duration = max(EF of all terminal activities)

Backward Pass (Calculate LS, LF):

1. For terminal activities: LF = Project duration, LS = LF - Duration
2. For each preceding activity: LF = min(LS of all successors)
3. LS = LF - Duration

Identify Critical Path: Activities where ES = LS (or EF = LF) have zero float and are on critical path.

4.3.1.4 Mathematical Formulation

For activity i with duration d_i:

$$\begin{aligned} ES_i &= \max_{j \in P(i)} EF_j \\ EF_i &= ES_i + d_i \\ LF_i &= \min_{j \in S(i)} LS_j \\ LS_i &= LF_i - d_i \\ \text{Float}_i &= LS_i - ES_i = LF_i - EF_i \end{aligned}$$

Where $P(i)$ are predecessors and $S(i)$ are successors.

4.4 Practice Problems: Critical Path Method (CPM)

4.4.1 The Systematic Approach: The "Pass" Method

To solve CPM problems without getting confused, we treat the project network like a circuit. We don't guess; we calculate.

The 4-Step Algorithm:

1. **Forward Pass (The Earliest Possible):**
   - Start at Time 0.
   - Move $Start \rightarrow End$.
   - **Rule:** $ES = \max(EF \text{ of predecessors})$.
   - *Logic:* You can't start until *all* your prerequisites are done.
2. **Backward Pass (The Latest Permissible):**
   - Start at the Project End Time.
   - Move $End \rightarrow Start$.
   - **Rule:** $LF = \min(LS \text{ of successors})$.
   - *Logic:* You must finish early enough so *everyone* waiting for you can start on time.
3. **Calculate Float (The Buffer):**
   - Float $= LS - ES$ (or $LF - EF$).

4. **Identify Critical Path:**
    - Any activity with **Zero Float** is Critical.

4.4.2 Problem 1: The "Hello World" of CPM

Scenario: A simple linear workflow with one parallel branch.

Activity Data:

Activity	Predecessors	Duration (Days)
A	-	3
B	A	4
C	A	2
D	B, C	5

Mathematical Solution:

Step 1: Forward Pass (Find ES, EF)

- **A:** Start at 0. Duration 3. $\rightarrow EF = 3$.
- **B:** Predecessor A (ends at 3). Start at 3. Duration 4. $\rightarrow EF = 7$.
- **C:** Predecessor A (ends at 3). Start at 3. Duration 2. $\rightarrow EF = 5$.
- **D:** Predecessors B (ends 7) and C (ends 5).
    - *Rule:* We must wait for the **slowest** one. $\max(7, 5) = 7$.
    - Start at 7. Duration 5. $\rightarrow EF = 12$.

Project Duration: 12 Days.

Step 2: Backward Pass (Find LF, LS)

- **D:** End at 12. Duration 5. $\rightarrow LS = 12 - 5 = 7$.
- **B:** Successor D (starts at 7). Must finish by 7. Duration 4. $\rightarrow LS = 7 - 4 = 3$.
- **C:** Successor D (starts at 7). Must finish by 7. Duration 2. $\rightarrow LS = 7 - 2 = 5$.
- **A:** Successors B (starts 3) and C (starts 5).
    - *Rule:* Must be done in time for the **earliest** requirement. $\min(3, 5) = 3$.
    - Must finish by 3. Duration 3. $\rightarrow LS = 0$.

Step 3 & 4: Float & Critical Path

Activity	Duration	ES	EF	LS	LF	Float (LS-ES)	Critical?
A	3	0	3	0	3	0	**Yes**
B	4	3	7	3	7	0	**Yes**
C	2	3	5	5	7	2	No
D	5	7	12	7	12	0	**Yes**

Critical Path: $A \to B \to D$

Python solution

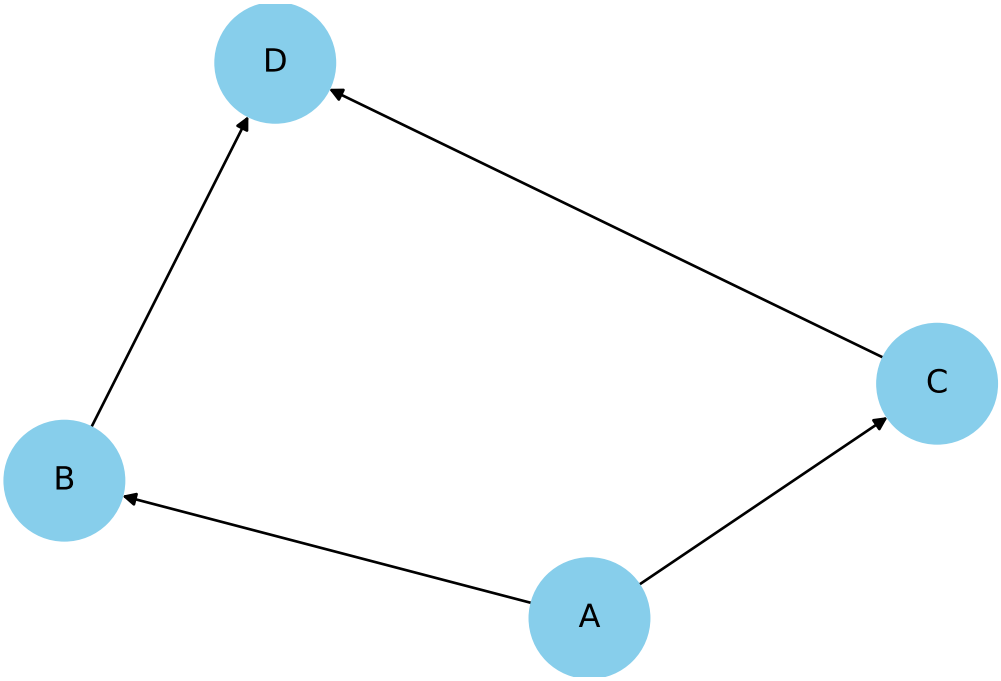
```
import networkx as nx
import pandas as pd

# 1. Define the Project Data
tasks = {
    'A': {'duration': 3, 'predecessors': []},
    'B': {'duration': 4, 'predecessors': ['A']},
    'C': {'duration': 2, 'predecessors': ['A']},
    'D': {'duration': 5, 'predecessors': ['B', 'C']}
}

# 2. Build the Network Graph
G = nx.DiGraph()
for task, data in tasks.items():
    G.add_node(task, duration=data['duration'])
    for pred in data['predecessors']:
        G.add_edge(pred, task)

# ------------------------------------------------------------
# 3. Forward Pass (Calculate Early Start & Early Finish)
# ------------------------------------------------------------
# We iterate in topological order to ensure predecessors are processed first
early_start = {}
early_finish = {}

for task in nx.topological_sort(G):
    # ES is max of predecessor's EF. If no predecessors, ES = 0
    predecessors = list(G.predecessors(task))
    if not predecessors:
        es = 0
```

```
    else:
        es = max(early_finish[p] for p in predecessors)

    duration = G.nodes[task]['duration']
    ef = es + duration

    early_start[task] = es
    early_finish[task] = ef

project_duration = max(early_finish.values())

# ------------------------------------------------------------
# 4. Backward Pass (Calculate Late Start & Late Finish)
# ------------------------------------------------------------
# We iterate in reverse topological order
late_start = {}
late_finish = {}

for task in reversed(list(nx.topological_sort(G))):
    successors = list(G.successors(task))

    # LF is min of successor's LS. If no successors, LF = Project Duration
    if not successors:
        lf = project_duration
    else:
        lf = min(late_start[s] for s in successors)

    duration = G.nodes[task]['duration']
    ls = lf - duration

    late_finish[task] = lf
    late_start[task] = ls

# ------------------------------------------------------------
# 5. Calculate Slack & Identify Critical Path
# ------------------------------------------------------------
slack = {}
critical_path = []

for task in tasks:
    s = late_start[task] - early_start[task]
    slack[task] = s
```

```
    if s == 0:
        critical_path.append(task)

# ------------------------------------------------------------
# 6. Display Results
# ------------------------------------------------------------
# Create a DataFrame for a clean CPM Table
df = pd.DataFrame({
    'Duration': [tasks[t]['duration'] for t in tasks],
    'ES (Early Start)': [early_start[t] for t in tasks],
    'EF (Early Finish)': [early_finish[t] for t in tasks],
    'LS (Late Start)': [late_start[t] for t in tasks],
    'LF (Late Finish)': [late_finish[t] for t in tasks],
    'Slack': [slack[t] for t in tasks],
    'Critical?': ['Yes' if s == 0 else 'No' for s in slack.values()]
}, index=tasks.keys())

print("--- CPM Schedule Table ---")
print(df)
print("-" * 30)
print(f"Total Project Duration: {project_duration} Days")
print(f"Critical Path: {' -> '.join(critical_path)}")

# Visualize the Network
import matplotlib.pyplot as plt
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True, node_size=2000, node_color='skyblue')
plt.show()
```

```
--- CPM Schedule Table ---
   Duration  ES (Early Start)  EF (Early Finish)  LS (Late Start)  \
A         3                 0                  3                0
B         4                 3                  7                3
C         2                 3                  5                5
D         5                 7                 12                7

   LF (Late Finish)  Slack Critical?
A                 3      0       Yes
B                 7      0       Yes
C                 7      2        No
D                12      0       Yes
------------------------------
```

```
Total Project Duration: 12 Days
Critical Path: A -> B -> D
```



4.4.3 Problem 2: The Bottleneck

Scenario: Software Development. Coding takes longer than Documentation.

Activity Data:

Activity	Description	Pred	Duration
S	Start Specs	-	2
C	Code Module	S	8
D	Write Docs	S	3
T	Testing	C, D	4

Mathematical Solution:

Forward Pass: 1. **S:** $0 \to 2$. 2. **C:** Starts after S(2). $2 + 8 = 10$. 3. **D:** Starts after S(2). $2 + 3 = 5$. 4. **T:** Waits for C(10) & D(5). $\max(10, 5) = 10$. $10 + 4 = 14$. *Project Ends: Day 14.*

Backward Pass: 1. **T:** Finish by 14. Start by $14 - 4 = 10$. 2. **C:** Must finish by T's start (10). Start by $10 - 8 = 2$. 3. **D:** Must finish by T's start (10). Start by $10 - 3 = 7$. 4. **S:** Successors C(starts 2) & D(starts 7). $\min(2, 7) = 2$. Start by $2 - 2 = 0$.

Float Calculation: * **Float C:** $LS(2) - ES(2) = 0$ (Critical) * **Float D:** $LS(7) - ES(2) = 5$ (Slack)

Interpretation: The Documentation team (D) can delay their start by up to 5 days without affecting the launch date. The Coding team (C) has zero wiggle room.

Critical Path: $S \to C \to T$

Python solution

```
import networkx as nx
import pandas as pd

# 1. Define the Project Data (Updated for Problem 2)
tasks = {
    'S': {'desc': 'Start Specs', 'duration': 2, 'predecessors': []},
    'C': {'desc': 'Code Module', 'duration': 8, 'predecessors': ['S']},
    'D': {'desc': 'Write Docs',  'duration': 3, 'predecessors': ['S']},
    'T': {'desc': 'Testing',     'duration': 4, 'predecessors': ['C', 'D']}
}

# 2. Build Graph
G = nx.DiGraph()
for task, data in tasks.items():
    G.add_node(task, duration=data['duration'], desc=data['desc'])
    for pred in data['predecessors']:
        G.add_edge(pred, task)

# 3. Forward Pass (Early Start/Finish)
early_start = {}
early_finish = {}
for task in nx.topological_sort(G):
    # ES = Max of predecessor EF
    predecessors = list(G.predecessors(task))
    es = max((early_finish[p] for p in predecessors), default=0)

    duration = G.nodes[task]['duration']
    early_start[task] = es
    early_finish[task] = es + duration
```

```
project_duration = max(early_finish.values())

# 4. Backward Pass (Late Start/Finish)
late_start = {}
late_finish = {}
for task in reversed(list(nx.topological_sort(G))):
    successors = list(G.successors(task))
    # LF = Min of successor LS
    lf = min((late_start[s] for s in successors), default=project_duration)

    duration = G.nodes[task]['duration']
    late_start[task] = lf - duration
    late_finish[task] = lf

# 5. Calculate Slack & Critical Path
slack = {}
critical_path = []
for task in tasks:
    s = late_start[task] - early_start[task]
    slack[task] = s
    if s == 0:
        critical_path.append(task)

# 6. Display Results
df = pd.DataFrame({
    'Description': [tasks[t]['desc'] for t in tasks],
    'Duration': [tasks[t]['duration'] for t in tasks],
    'ES': [early_start[t] for t in tasks],
    'EF': [early_finish[t] for t in tasks],
    'LS': [late_start[t] for t in tasks],
    'LF': [late_finish[t] for t in tasks],
    'Slack': [slack[t] for t in tasks],
    'Critical': ['Yes' if s == 0 else 'No' for s in slack.values()]
}, index=tasks.keys())

print("--- CPM Schedule Analysis: The Bottleneck ---")
print(df[['Description', 'Duration', 'ES', 'EF', 'Slack', 'Critical']])
print("-" * 60)
print(f"Total Project Duration: {project_duration} Days")
print(f"Critical Path: {' -> '.join(critical_path)}")
```

```
--- CPM Schedule Analysis: The Bottleneck ---
```

```
   Description  Duration  ES  EF  Slack Critical
S  Start Specs         2   0   2      0      Yes
C  Code Module         8   2  10      0      Yes
D   Write Docs         3   2   5      5       No
T      Testing         4  10  14      0      Yes
------------------------------------------------------------
Total Project Duration: 14 Days
Critical Path: S -> C -> T
```

4.4.4 Problem 3: The "Merge" Challenge

Scenario: Construction. Notice how Activity F depends on three things.

Activity Data:

Act	Pred	Time
A	-	5
B	A	3
C	A	4
D	A	6
E	B	2
F	C, D, E	3

Mathematical Solution:

Step 1: The Merge (Node F) * First, compute B, C, D (all start after A finishes at 5).

```
- **B:** $5 + 3 = 8$. (E follows B: $8 + 2 = 10$).
- **C:** $5 + 4 = 9$.
- **D:** $5 + 6 = 11$.
```

- **F** depends on C(9), D(11), E(10).
- **F's Start:** $\max(9, 11, 10) = 11$.
- **F's Finish:** $11 + 3 = 14$.

Step 2: Backward from Merge

- **F:** $LS = 11$.
- Now F pushes back on C, D, and E. They ALL must finish by 11.
  - **D:** $LF = 11$. Duration 6. $LS = 5$.
  - **C:** $LF = 11$. Duration 4. $LS = 7$.

– **E:** $LF = 11$. Duration 2. $LS = 9$. (This pushes back on B).

Float Analysis:

- **Path A-D-F:** $5 + 6 + 3 = 14$. (Length 14). Float $= 0$.
- **Path A-C-F:** $5 + 4 + 3 = 12$. (Length 12). Float $= 2$.
- **Path A-B-E-F:** $5 + 3 + 2 + 3 = 13$. (Length 13). Float $= 1$.

Critical Path: $A \to D \to F$

Python solution

```
import networkx as nx
import pandas as pd

# 1. Define the Project Data (Problem 3: The Merge)
tasks = {
    'A': {'duration': 5, 'predecessors': []},
    'B': {'duration': 3, 'predecessors': ['A']},
    'C': {'duration': 4, 'predecessors': ['A']},
    'D': {'duration': 6, 'predecessors': ['A']},
    'E': {'duration': 2, 'predecessors': ['B']},
    'F': {'duration': 3, 'predecessors': ['C', 'D', 'E']}
}

# 2. Build Graph
G = nx.DiGraph()
for task, data in tasks.items():
    G.add_node(task, duration=data['duration'])
    for pred in data['predecessors']:
        G.add_edge(pred, task)

# 3. Forward Pass (Early Start/Finish)
early_start = {}
early_finish = {}
# Topological sort ensures we process parents before children
for task in nx.topological_sort(G):
    predecessors = list(G.predecessors(task))
    # ES is Max of predecessors' EF
    es = max((early_finish[p] for p in predecessors), default=0)

    duration = G.nodes[task]['duration']
    early_start[task] = es
    early_finish[task] = es + duration
```

```
project_duration = max(early_finish.values())

# 4. Backward Pass (Late Start/Finish)
late_start = {}
late_finish = {}
for task in reversed(list(nx.topological_sort(G))):
    successors = list(G.successors(task))
    # LF is Min of successors' LS
    lf = min((late_start[s] for s in successors), default=project_duration)

    duration = G.nodes[task]['duration']
    late_start[task] = lf - duration
    late_finish[task] = lf

# 5. Calculate Slack & Critical Path
slack = {}
critical_path = []
for task in tasks:
    s = late_start[task] - early_start[task]
    slack[task] = s
    if s == 0:
        critical_path.append(task)

# 6. Display Results
df = pd.DataFrame({
    'Duration': [tasks[t]['duration'] for t in tasks],
    'ES': [early_start[t] for t in tasks],
    'EF': [early_finish[t] for t in tasks],
    'LS': [late_start[t] for t in tasks],
    'LF': [late_finish[t] for t in tasks],
    'Slack': [slack[t] for t in tasks],
    'Critical': ['Yes' if s == 0 else 'No' for s in slack.values()]
}, index=tasks.keys())

print("--- CPM Schedule: The Merge Challenge ---")
print(df)
print("-" * 60)
print(f"Total Project Duration: {project_duration} Days")
print(f"Critical Path: {' -> '.join(critical_path)}")
```

```
--- CPM Schedule: The Merge Challenge ---
```

```
   Duration  ES  EF  LS  LF  Slack Critical
A         5   0   5   0   5      0      Yes
B         3   5   8   6   9      1       No
C         4   5   9   7  11      2       No
D         6   5  11   5  11      0      Yes
E         2   8  10   9  11      1       No
F         3  11  14  11  14      0      Yes
------------------------------------------------------------
Total Project Duration: 14 Days
Critical Path: A -> D -> F
```

4.4.5 Problem 4: Complex Dependencies

Scenario: A tech product launch.

	Act	Desc	Pred	Time
A	Market Research		-	10
B	Product Design		A	20
C	Ad Campaign		A	15
D	Prototype		B	10
E	Manufacture		D	30
F	Launch Event		C, E	5

Mathematical Solution:

Trace Table:

Node	Pred	ES	EF (=ES+Dur)	Succ	LF	LS (=LF-Dur)	Float
A	-	0	10	B, C	10	0	0
B	A	10	30	D	30	10	0
C	A	10	25	F	70	55	**45**
D	B	30	40	E	40	30	0
E	D	40	70	F	70	40	0
F	C, E	**70**	75	-	75	70	0

Calculations to Watch:

- For **F**: Predecessors C(25) and E(70). $\max(25, 70) = 70$.
- For **C**: Successor F requires start at 70. Duration 15. $LS = 70 - 15 = 55$.

- Float for C: $55 - 10 = 45$.
- *Insight:* The Ad Campaign (C) is wildly non-critical. You have 45 days of slack!

Critical Path: $A \to B \to D \to E \to F$

Python solution

```
import networkx as nx
import pandas as pd

# 1. Define the Project Data (Problem 4: Tech Launch)
tasks = {
    'A': {'desc': 'Market Research', 'duration': 10, 'predecessors': []},
    'B': {'desc': 'Product Design',  'duration': 20, 'predecessors': ['A']},
    'C': {'desc': 'Ad Campaign',     'duration': 15, 'predecessors': ['A']},
    'D': {'desc': 'Prototype',       'duration': 10, 'predecessors': ['B']},
    'E': {'desc': 'Manufacture',     'duration': 30, 'predecessors': ['D']},
    'F': {'desc': 'Launch Event',    'duration': 5,  'predecessors': ['C', 'E']}
}

# 2. Build Graph
G = nx.DiGraph()
for task, data in tasks.items():
    G.add_node(task, duration=data['duration'], desc=data['desc'])
    for pred in data['predecessors']:
        G.add_edge(pred, task)

# 3. Forward Pass (Early Start/Finish)
early_start = {}
early_finish = {}
for task in nx.topological_sort(G):
    predecessors = list(G.predecessors(task))
    es = max((early_finish[p] for p in predecessors), default=0)

    duration = G.nodes[task]['duration']
    early_start[task] = es
    early_finish[task] = es + duration

project_duration = max(early_finish.values())

# 4. Backward Pass (Late Start/Finish)
late_start = {}
late_finish = {}
```

```
for task in reversed(list(nx.topological_sort(G))):
    successors = list(G.successors(task))
    lf = min((late_start[s] for s in successors), default=project_duration)

    duration = G.nodes[task]['duration']
    late_start[task] = lf - duration
    late_finish[task] = lf

# 5. Calculate Slack & Critical Path
slack = {}
critical_path = []
for task in tasks:
    s = late_start[task] - early_start[task]
    slack[task] = s
    if s == 0:
        critical_path.append(task)

# 6. Display Results
df = pd.DataFrame({
    'Description': [tasks[t]['desc'] for t in tasks],
    'Duration': [tasks[t]['duration'] for t in tasks],
    'ES': [early_start[t] for t in tasks],
    'EF': [early_finish[t] for t in tasks],
    'LS': [late_start[t] for t in tasks],
    'LF': [late_finish[t] for t in tasks],
    'Slack': [slack[t] for t in tasks],
    'Critical': ['Yes' if s == 0 else 'No' for s in slack.values()]
}, index=tasks.keys())

print("--- CPM Analysis: Tech Product Launch ---")
print(df[['Description', 'Duration', 'ES', 'EF', 'Slack', 'Critical']])
print("-" * 65)
print(f"Total Project Duration: {project_duration} Days")
print(f"Critical Path: {' -> '.join(critical_path)}")
```

```
--- CPM Analysis: Tech Product Launch ---
       Description  Duration  ES  EF  Slack Critical
A  Market Research        10   0  10      0      Yes
B   Product Design        20  10  30      0      Yes
C      Ad Campaign        15  10  25     45       No
D        Prototype        10  30  40      0      Yes
E      Manufacture        30  40  70      0      Yes
```

```
F     Launch Event          5  70  75      0      Yes
-----------------------------------------------------------------
Total Project Duration: 75 Days
Critical Path: A -> B -> D -> E -> F
```

4.4.6 Problem 5: The "Exam Style" Tabular Problem

Scenario: Complete the table to find the project parameters.

Given Data:

- A (4), B (3) start at 0.
- C (2) depends on A.
- D (5) depends on A.
- E (6) depends on B.
- F (3) depends on C, D, E.

Mathematical Solution:

Forward Pass:

1. **A:** $0 \to 4$.
2. **B:** $0 \to 3$.
3. **C:** After A(4). $4 \to 6$.
4. **D:** After A(4). $4 \to 9$.
5. **E:** After B(3). $3 \to 9$.
6. **F:** After C(6), D(9), E(9). $\max(6, 9, 9) = 9$. $9 \to 12$.

Backward Pass:

1. **F:** End 12. Start 9.
2. **C:** Need by 9. End 9. Start $9 - 2 = 7$.
3. **D:** Need by 9. End 9. Start $9 - 5 = 4$.
4. **E:** Need by 9. End 9. Start $9 - 6 = 3$.
5. **A:** Succ C(start 7), D(start 4). $\min(7, 4) = 4$. End 4. Start 0.
6. **B:** Succ E(start 3). End 3. Start 0.

Results Table:

Activity	ES	EF	LS	LF	Float	Critical?
A	0	4	0	4	0	**YES**
B	0	3	0	3	0	**YES**
C	4	6	7	9	3	No
D	4	9	4	9	0	**YES**

Activity	ES	EF	LS	LF	Float	Critical?
E	3	9	3	9	0	**YES**
F	9	12	9	12	0	**YES**

Observation:

This project has **Two Critical Paths**:

1. $A \to D \to F$ (Length $4 + 5 + 3 = 12$)
2. $B \to E \to F$ (Length $3 + 6 + 3 = 12$)

Both paths are bottlenecks. Delays in either A OR B will delay the project.

Python solution

```
import networkx as nx
import pandas as pd

# 1. Define the Project Data (Problem 5)
tasks = {
    'A': {'duration': 4, 'predecessors': []},
    'B': {'duration': 3, 'predecessors': []},
    'C': {'duration': 2, 'predecessors': ['A']},
    'D': {'duration': 5, 'predecessors': ['A']},
    'E': {'duration': 6, 'predecessors': ['B']},
    'F': {'duration': 3, 'predecessors': ['C', 'D', 'E']}
}

# 2. Build Graph
G = nx.DiGraph()
for task, data in tasks.items():
    G.add_node(task, duration=data['duration'])
    for pred in data['predecessors']:
        G.add_edge(pred, task)

# 3. Forward Pass (Early Start/Finish)
early_start = {}
early_finish = {}
for task in nx.topological_sort(G):
    predecessors = list(G.predecessors(task))
    # If no preds, start at 0. Else max of preds' finish.
    es = max((early_finish[p] for p in predecessors), default=0)
```

```
    duration = G.nodes[task]['duration']
    early_start[task] = es
    early_finish[task] = es + duration

project_duration = max(early_finish.values())

# 4. Backward Pass (Late Start/Finish)
late_start = {}
late_finish = {}
for task in reversed(list(nx.topological_sort(G))):
    successors = list(G.successors(task))
    # If no successors, finish at Project Duration. Else min of succs' start.
    lf = min((late_start[s] for s in successors), default=project_duration)

    duration = G.nodes[task]['duration']
    late_start[task] = lf - duration
    late_finish[task] = lf

# 5. Calculate Slack & Identify Critical Paths
slack = {}
critical_path_tasks = []
for task in tasks:
    s = late_start[task] - early_start[task]
    slack[task] = s
    if s == 0:
        critical_path_tasks.append(task)

# 6. Display Results
df = pd.DataFrame({
    'Duration': [tasks[t]['duration'] for t in tasks],
    'ES': [early_start[t] for t in tasks],
    'EF': [early_finish[t] for t in tasks],
    'LS': [late_start[t] for t in tasks],
    'LF': [late_finish[t] for t in tasks],
    'Slack': [slack[t] for t in tasks],
    'Critical': ['Yes' if s == 0 else 'No' for s in slack.values()]
}, index=tasks.keys())

print("--- CPM Tabular Solution ---")
print(df)
print("-" * 60)
print(f"Total Project Duration: {project_duration} Days")
```

```
print(f"Critical Tasks: {', '.join(critical_path_tasks)}")
```

```
--- CPM Tabular Solution ---
   Duration  ES  EF  LS  LF  Slack Critical
A         4   0   4   0   4      0      Yes
B         3   0   3   0   3      0      Yes
C         2   4   6   7   9      3       No
D         5   4   9   4   9      0      Yes
E         6   3   9   3   9      0      Yes
F         3   9  12   9  12      0      Yes
------------------------------------------------------------
Total Project Duration: 12 Days
Critical Tasks: A, B, D, E, F
```

4.4.7 Sample Problem

Problem Statement: Solve the library construction problem from Section 3.1 using CPM.

Solution:

Forward Pass:

- A: ES=0, EF=2
- B: ES=2, EF=6
- C: ES=6, EF=12
- D: ES=12, EF=15
- E: ES=12, EF=15
- F: ES=15, EF=19
- G: ES=12, EF=14
- H: ES=19, EF=20

Backward Pass:

- H: LF=20, LS=19
- F: LF=19, LS=15
- G: LF=19, LS=17
- D: LF=15, LS=12
- E: LF=15, LS=12
- C: LF=12, LS=6
- B: LF=6, LS=2
- A: LF=2, LS=0

Critical Path: A → B → C → D → F → H (Float = 0 for all) **Project Duration:** 20 weeks

Non-critical Activity: G has float of 3 weeks (LS=17, ES=14)

4.4.8 Sample Problem

Problem Statement: A campus renovation project has the following activities with crashing costs (cost to reduce duration by 1 week):

Activity	Normal Time	Crash Time	Normal Cost	Crash Cost	Predecessors
A	3	2	$1000	$1500	-
B	4	3	$2000	$2600	A
C	5	4	$1500	$2000	A
D	6	4	$3000	$4200	B, C

Find the minimum cost schedule to complete in 10 weeks.

Solution Approach: Use CPM with time-cost trade-off analysis (crashing). Calculate cost slope = (Crash Cost - Normal Cost)/(Normal Time - Crash Time) and crash activities on critical path starting with lowest cost slope.

Python solution

```
import networkx as nx
import pandas as pd
from itertools import combinations

# ------------------------------------------------------------
# 1. Setup Project Data
# ------------------------------------------------------------
# Format: {Activity: {NormalT, CrashT, NormalC, CrashC, Predecessors}}
data = {
    'A': {'nT': 3, 'cT': 2, 'nC': 1000, 'cC': 1500, 'pred': []},
    'B': {'nT': 4, 'cT': 3, 'nC': 2000, 'cC': 2600, 'pred': ['A']},
    'C': {'nT': 5, 'cT': 4, 'nC': 1500, 'cC': 2000, 'pred': ['A']},
    'D': {'nT': 6, 'cT': 4, 'nC': 3000, 'cC': 4200, 'pred': ['B', 'C']}
}

TARGET_DURATION = 10

# ------------------------------------------------------------
```

```python
# 2. Helper Functions
# ------------------------------------------------------------

def calculate_slope(task):
    """Calculates Cost Slope = (Crash Cost - Normal Cost) / (Normal Time - Crash Time)"""
    props = data[task]
    if props['nT'] == props['cT']: return float('inf') # Cannot crash
    return (props['cC'] - props['nC']) / (props['nT'] - props['cT'])

def get_project_duration(current_times):
    """Runs CPM Forward Pass to find Project Duration"""
    G = nx.DiGraph()
    for task, props in data.items():
        G.add_node(task, duration=current_times[task])
        for p in props['pred']:
            G.add_edge(p, task)

    # Calculate Earliest Finish (EF)
    early_finish = {}
    for task in nx.topological_sort(G):
        preds = list(G.predecessors(task))
        es = max((early_finish[p] for p in preds), default=0)
        early_finish[task] = es + current_times[task]

    return max(early_finish.values())

# ------------------------------------------------------------
# 3. Initialization
# ------------------------------------------------------------
# Current state of durations (Starts at Normal Time)
current_times = {t: data[t]['nT'] for t in data}
current_cost = sum(data[t]['nC'] for t in data)
history = []

# Calculate Slopes
slopes = {t: calculate_slope(t) for t in data}

# ------------------------------------------------------------
# 4. Iterative Crashing Loop
# ------------------------------------------------------------
print("--- Crashing Analysis Log ---")
```

```
while True:
    duration = get_project_duration(current_times)
    history.append({'Duration': duration, 'Total Cost': current_cost, 'Action': 'Status Chec

    if duration <= TARGET_DURATION:
        print(f"Target of {TARGET_DURATION} weeks reached!")
        break

    # Find the Best Move (Cheapest way to reduce duration by 1 unit)
    best_move = None
    min_move_cost = float('inf')

    # Strategy A: Try crashing single tasks
    candidates = [t for t in data if current_times[t] > data[t]['cT']]

    for task in candidates:
        # Simulate Crash
        current_times[task] -= 1
        new_dur = get_project_duration(current_times)
        current_times[task] += 1 # Revert

        # If duration decreased, this is a valid move
        if new_dur < duration:
            cost = slopes[task]
            if cost < min_move_cost:
                min_move_cost = cost
                best_move = [task]

    # Strategy B: If single tasks fail (due to parallel critical paths), try pairs
    if best_move is None:
        # We need to crash two parallel tasks simultaneously (e.g., B and C)
        for t1, t2 in combinations(candidates, 2):
            current_times[t1] -= 1
            current_times[t2] -= 1
            new_dur = get_project_duration(current_times)
            current_times[t1] += 1
            current_times[t2] += 1

            if new_dur < duration:
                cost = slopes[t1] + slopes[t2]
                if cost < min_move_cost:
                    min_move_cost = cost
```

```
                    best_move = [t1, t2]

    # Apply the Best Move
    if best_move:
        actions_str = ""
        for task in best_move:
            current_times[task] -= 1
            current_cost += slopes[task]
            actions_str += f"Crash {task} (-1 wk, +${slopes[task]}); "

        print(f"Current Duration: {duration} wks. Action: {actions_str}")
    else:
        print("Cannot crash further (Max limit reached).")
        break

# ------------------------------------------------------------
# 5. Final Output
# ------------------------------------------------------------
df_res = pd.DataFrame(history)
print("\n--- Final Project Schedule ---")
print(f"Final Duration: {get_project_duration(current_times)} weeks")
print(f"Final Total Cost: ${current_cost}")
print(f"Durations: {current_times}")
print("-" * 40)
print(df_res)
```

```
--- Crashing Analysis Log ---
Current Duration: 14 wks. Action: Crash A (-1 wk, +$500.0);
Current Duration: 13 wks. Action: Crash C (-1 wk, +$500.0);
Current Duration: 12 wks. Action: Crash D (-1 wk, +$600.0);
Current Duration: 11 wks. Action: Crash D (-1 wk, +$600.0);
Target of 10 weeks reached!

--- Final Project Schedule ---
Final Duration: 10 weeks
Final Total Cost: $9700.0
Durations: {'A': 2, 'B': 4, 'C': 4, 'D': 4}
----------------------------------------
   Duration  Total Cost        Action
0        14      7500.0  Status Check
1        13      8000.0  Status Check
2        12      8500.0  Status Check
```

```
3         11       9100.0  Status Check
4         10       9700.0  Status Check
```

4.4.9 Review Questions

1. Explain why activities on the critical path have zero float. What implications does this have for project management?
2. How does CPM help in resource allocation for campus construction projects?
3. Given a project network, how would you determine all possible critical paths if multiple exist?
4. What is the significance of float/slack time in project scheduling? How can it be utilized effectively?
5. Design a CPM analysis for planning a campus festival. Identify what would constitute critical activities.

4.5 Program Evaluation and Review Technique (PERT)

4.5.1 Theoretical Foundation

4.5.1.1 What is PERT?

CPM assumes we know exactly how long tasks take. But in reality, construction crews get sick, materials arrive late, and code has bugs. PERT extends CPM by incorporating probabilistic time estimates to handle uncertainty in activity durations.

4.5.1.2 Three-Point Time Estimates

Each activity has three duration estimates:

1. **Optimistic Time (a)**: Minimum possible time under ideal conditions
2. **Most Likely Time (m)**: Most probable time under normal conditions
3. **Pessimistic Time (b)**: Maximum possible time under worst conditions

4.5.1.3 PERT Calculations

Expected Time (te):

$$t_e = \frac{a + 4m + b}{6}$$

Variance (²):

$$\sigma^2 = \left(\frac{b - a}{6}\right)^2$$

Standard Deviation ( ):

$$\sigma = \frac{b - a}{6}$$

> Why divide by 6?
>
> This approximation assumes the range from a to b covers 6 standard deviations ($\pm 3\sigma$) of a normal distribution, which covers 99.7% of all outcomes. It is a brilliant statistical shortcut!

4.5.2 Probability of Deadline Success

Once we have the expected duration and variance for the Critical Path, we can use the **Central Limit Theorem**. It tells us the total project time will follow a Normal Distribution (Bell Curve).

4.5.2.1 Probability Analysis

For the entire project:

- **Expected Project Duration**: Sum of te along critical path
- **Project Variance**: Sum of variances along critical path
- **Project Standard Deviation**: Square root of project variance

We can calculate the **Z-score**:

Using normal distribution (Central Limit Theorem):

$$Z = \frac{T - T_E}{\sigma_p}$$

Where:

- T: Target completion time
- T_E: Expected project duration
- σ_p: Project standard deviation
- Z: Standard normal variable Or

$$Z = \frac{\text{Target Time} - \text{Expected Time}}{\text{Project Standard Deviation}}$$

This Z value tells us the precise probability of meeting a deadline. For example, if $Z = 1.65$, there is a 95% chance you finish on time. This is how professional engineers manage risk. **Probability of Completion:**

4.5.2.2 PERT vs CPM Comparison

Aspect	CPM	PERT
Time Estimates	Deterministic	Probabilistic
Focus	Time-Cost trade-offs	Time uncertainty
Best For	Well-defined projects	Research/development
Output	Critical path, floats	Completion probabilities

4.5.3 Sample Problem

Problem Statement: For the library construction project, add probabilistic time estimates:

Activity	a	m	b	Predecessors
A	1	2	3	-
B	3	4	5	A
C	5	6	7	B
D	2	3	4	C
E	2	3	4	C
F	3	4	5	D, E
G	1	2	3	C
H	1	1	1	F, G

Calculate:

1. Expected project duration
2. Probability of completing in 22 weeks
3. 95% confidence interval for completion

Solution:

Step 1: Calculate te for each activity:

- A: (1+8+3)/6 = 2
- B: (3+16+5)/6 = 4
- C: (5+24+7)/6 = 6
- D: (2+12+4)/6 = 3
- E: (2+12+4)/6 = 3
- F: (3+16+5)/6 = 4
- G: (1+8+3)/6 = 2
- H: (1+4+1)/6 = 1

Step 2: Identify critical path (same as CPM): A-B-C-D-F-H

Expected duration: 2+4+6+3+4+1 = 20 weeks

Step 3: Calculate variances along critical path:

- A: ((3-1)/6)² = 0.11
- B: ((5-3)/6)² = 0.11
- C: ((7-5)/6)² = 0.11
- D: ((4-2)/6)² = 0.11
- F: ((5-3)/6)² = 0.11
- H: ((1-1)/6)² = 0

Total variance: 0.55

Standard deviation: √0.55 = 0.74 weeks

Step 4: Probability of completing in 22 weeks:

Z = (22-20)/0.74 = 2.70, P(Z 2.70) 0.9965 → 99.65% probability

Step 5: 95% confidence interval:

Z for 95% = 1.96, Interval: 20 ± 1.96×0.74 = [18.55, 21.45] weeks

Python solution

```
import math
from collections import deque

def solve_pert_problem():
    # Activity data: a (optimistic), m (most likely), b (pessimistic), predecessors
    activities = {
        'A': {'a': 1, 'm': 2, 'b': 3, 'pred': []},
        'B': {'a': 3, 'm': 4, 'b': 5, 'pred': ['A']},
```

```
        'C': {'a': 5, 'm': 6, 'b': 7, 'pred': ['B']},
        'D': {'a': 2, 'm': 3, 'b': 4, 'pred': ['C']},
        'E': {'a': 2, 'm': 3, 'b': 4, 'pred': ['C']},
        'F': {'a': 3, 'm': 4, 'b': 5, 'pred': ['D', 'E']},
        'G': {'a': 1, 'm': 2, 'b': 3, 'pred': ['C']},
        'H': {'a': 1, 'm': 1, 'b': 1, 'pred': ['F', 'G']}
    }

    # Calculate expected time and variance for each activity
    print("Activity Analysis:")
    print("=" * 60)
    print(f"{'Activity':<10} {'Optimistic (a)':<15} {'Most Likely (m)':<15} {'Pessimistic (b)
    print("-" * 90)

    for activity, data in activities.items():
        a, m, b = data['a'], data['m'], data['b']
        # PERT formula: Expected time = (a + 4m + b) / 6
        te = (a + 4 * m + b) / 6
        # Variance = ((b - a) / 6)^2
        variance = ((b - a) / 6) ** 2
        activities[activity]['te'] = round(te, 2)
        activities[activity]['variance'] = round(variance, 2)
        print(f"{activity:<10} {a:<15} {m:<15} {b:<15} {te:<20.2f} {variance:<15.4f}")

    print()

    # Forward pass: Calculate earliest start (ES) and earliest finish (EF)
    print("Forward Pass Calculation:")
    print("=" * 60)
    print(f"{'Activity':<10} {'ES':<10} {'EF':<10}")
    print("-" * 30)

    # Initialize ES and EF for all activities
    for activity in activities:
        activities[activity]['ES'] = 0
        activities[activity]['EF'] = 0

    # Perform topological sort using Kahn's algorithm
    in_degree = {activity: len(activities[activity]['pred']) for activity in activities}
    queue = deque([activity for activity in activities if in_degree[activity] == 0])

    while queue:
```

```
        current = queue.popleft()
        current_data = activities[current]

        # Calculate ES: max of EF of all predecessors
        if current_data['pred']:
            current_data['ES'] = max(activities[pred]['EF'] for pred in current_data['pred'])
        else:
            current_data['ES'] = 0

        # Calculate EF: ES + expected time
        current_data['EF'] = current_data['ES'] + current_data['te']

        print(f"{current:<10} {current_data['ES']:<10.2f} {current_data['EF']:<10.2f}")

        # Update in-degree for successors and add to queue if in-degree becomes 0
        for activity in activities:
            if current in activities[activity]['pred']:
                in_degree[activity] -= 1
                if in_degree[activity] == 0:
                    queue.append(activity)

    # Find project duration (maximum EF)
    project_duration = max(activities[activity]['EF'] for activity in activities)
    print(f"\nExpected Project Duration: {project_duration:.2f} weeks")

    # Find critical path
    print("\nCritical Path Analysis:")
    print("=" * 60)

    # Backward pass: Calculate latest start (LS) and latest finish (LF)
    for activity in activities:
        activities[activity]['LF'] = project_duration
        activities[activity]['LS'] = project_duration

    # Process activities in reverse order
    reverse_order = list(activities.keys())[::-1]

    for activity in reverse_order:
        current_data = activities[activity]

        # Find minimum LS of all successors
        successors = [a for a in activities if activity in activities[a]['pred']]
```

```
        if successors:
            current_data['LF'] = min(activities[s]['LS'] for s in successors)
        else:
            current_data['LF'] = project_duration

        current_data['LS'] = current_data['LF'] - current_data['te']

    # Identify critical activities and calculate path variance
    print(f"{'Activity':<10} {'Slack':<10} {'Critical?':<15} {'Variance':<10}")
    print("-" * 45)

    critical_path = []
    path_variance = 0

    for activity in activities:
        slack = activities[activity]['LS'] - activities[activity]['ES']
        is_critical = abs(slack) < 0.01  # Considering floating point errors

        print(f"{activity:<10} {slack:<10.2f} {'Yes' if is_critical else 'No':<15} {activitie

        if is_critical:
            critical_path.append(activity)
            path_variance += activities[activity]['variance']

    print(f"\nCritical Path: {' -> '.join(critical_path)}")
    print(f"Critical Path Variance: {path_variance:.4f}")
    print(f"Critical Path Standard Deviation: {math.sqrt(path_variance):.4f}")

    # Calculate probability of completing in 22 weeks
    print("\n" + "=" * 60)
    print("PROBABILITY ANALYSIS")
    print("=" * 60)

    target_time = 22
    z_score = (target_time - project_duration) / math.sqrt(path_variance)

    print(f"Target completion time: {target_time} weeks")
    print(f"Expected completion time: {project_duration:.2f} weeks")
    print(f"Standard deviation of critical path: {math.sqrt(path_variance):.4f}")
    print(f"Z-score: {z_score:.4f}")

    # Calculate probability using normal distribution
```

```
    from scipy.stats import norm

    probability = norm.cdf(z_score)
    print(f"Probability of completing in {target_time} weeks: {probability:.4f} or {probabili

    # Calculate 95% confidence interval
    print("\n" + "=" * 60)
    print("95% CONFIDENCE INTERVAL")
    print("=" * 60)

    # Z-value for 95% confidence is approximately 1.96
    z_95 = 1.96
    margin_of_error = z_95 * math.sqrt(path_variance)

    lower_bound = project_duration - margin_of_error
    upper_bound = project_duration + margin_of_error

    print(f"95% Confidence Interval for completion:")
    print(f"Lower bound: {lower_bound:.2f} weeks")
    print(f"Upper bound: {upper_bound:.2f} weeks")
    print(f"We can be 95% confident that the project will take between {lower_bound:.2f} and

    # Summary
    print("\n" + "=" * 60)
    print("SUMMARY")
    print("=" * 60)
    print(f"1. Expected project duration: {project_duration:.2f} weeks")
    print(f"2. Probability of completing in {target_time} weeks: {probability*100:.2f}%")
    print(f"3. 95% confidence interval: [{lower_bound:.2f}, {upper_bound:.2f}] weeks")


if __name__ == "__main__":
    solve_pert_problem()
```

```
Activity Analysis:
============================================================
Activity   Optimistic (a)  Most Likely (m) Pessimistic (b) Expected Time (te)   Variance ( ²)
------------------------------------------------------------------------------------------
A          1               2               3               2.00                0.1111
B          3               4               5               4.00                0.1111
C          5               6               7               6.00                0.1111
D          2               3               4               3.00                0.1111
```

```
E          2               3               4               3.00                0.1111
F          3               4               5               4.00                0.1111
G          1               2               3               2.00                0.1111
H          1               1               1               1.00                0.0000

Forward Pass Calculation:
============================================================
Activity   ES         EF
------------------------------
A          0.00       2.00
B          2.00       6.00
C          6.00       12.00
D          12.00      15.00
E          12.00      15.00
G          12.00      14.00
F          15.00      19.00
H          19.00      20.00

Expected Project Duration: 20.00 weeks

Critical Path Analysis:
============================================================
Activity   Slack      Critical?       Variance
----------------------------------------------
A          0.00       Yes             0.1100
B          0.00       Yes             0.1100
C          0.00       Yes             0.1100
D          0.00       Yes             0.1100
E          0.00       Yes             0.1100
F          0.00       Yes             0.1100
G          5.00       No              0.1100
H          0.00       Yes             0.0000

Critical Path: A -> B -> C -> D -> E -> F -> H
Critical Path Variance: 0.6600
Critical Path Standard Deviation: 0.8124

============================================================
PROBABILITY ANALYSIS
============================================================
Target completion time: 22 weeks
Expected completion time: 20.00 weeks
Standard deviation of critical path: 0.8124
```

```
Z-score: 2.4618
Probability of completing in 22 weeks: 0.9931 or 99.31%

============================================================
95% CONFIDENCE INTERVAL
============================================================
95% Confidence Interval for completion:
Lower bound: 18.41 weeks
Upper bound: 21.59 weeks
We can be 95% confident that the project will take between 18.41 and 21.59 weeks

============================================================
SUMMARY
============================================================
1. Expected project duration: 20.00 weeks
2. Probability of completing in 22 weeks: 99.31%
3. 95% confidence interval: [18.41, 21.59] weeks
```

Problem Statement: A campus research project has activity D with estimates: a=4, m=5, b=12 weeks. Calculate the expected time and standard deviation. What does the large range (b-a) indicate?

Solution:

te = (4+20+12)/6 = 6 weeks = (12-4)/6 = 1.33 weeks

Interpretation: Large uncertainty in this activity duration. Should be monitored closely.

4.5.4 Review Questions

1. Explain why PERT uses the formula (a+4m+b)/6 for expected time rather than a simple average.
2. How does the assumption of normal distribution for project completion time rely on the Central Limit Theorem?
3. Calculate the probability of completing a project with expected duration 45 days and standard deviation 5 days within 50 days.
4. What managerial insights can be gained from PERT probability analysis that are not available from deterministic CPM?
5. Design a PERT analysis for planning a campus conference. Which activities would likely have the highest uncertainty and why?

4.6 Gantt Charts and Project Visualization

While CPM and PERT are the *brains* of the operation, the **Gantt Chart** is the *face*. It is a horizontal bar chart that maps activities against time.

4.6.1 Why do we need this?

Imagine explaining "Nodes and Edges" to a University Dean or a Construction Foreman. They might not understand graph theory, but they understand a bar chart.

- It visualizes **Parallelism**: "Look, the electricians and plumbers are working at the same time."
- It highlights **blockers**: "We cannot paint until the drywall is up."

4.6.2 Theoretical Foundation

4.6.2.1 What is a Gantt Chart?

A Gantt chart is a bar chart that visually represents a project schedule, showing:

- Activities/tasks on vertical axis
- Timeline on horizontal axis
- Bars showing activity durations and relationships

4.6.2.2 Components of Gantt Charts

1. **Task Bars**: Horizontal bars representing activity durations
2. **Milestone Markers**: Diamond symbols for key events
3. **Dependency Lines**: Arrows showing precedence relationships
4. **Progress Bars**: Shaded portions showing completed work
5. **Resource Assignments**: Color coding or labels for resources

4.6.2.3 Creating Gantt Charts from CPM/PERT

Steps:

1. List all activities with durations and dependencies
2. Calculate ES, EF, LS, LF using CPM
3. Draw timeline on horizontal axis
4. Plot each activity as bar from ES to EF

5. Add dependencies as arrows between bars
6. Highlight critical path activities
7. Add milestones and resources

4.6.2.4 Advanced Features

- **Baseline Schedule**: Original plan for comparison
- **Actual Progress**: Overlay of completed work
- **Resource Loading**: Show resource utilization over time
- **Critical Path Highlighting**: Color-code critical activities
- **Slack/Float Representation**: Show available flexibility

4.6.2.5 Benefits for Campus Projects

1. **Visual Communication**: Easy to understand for stakeholders
2. **Progress Tracking**: Monitor actual vs planned
3. **Resource Management**: Identify overallocation
4. **Schedule Adjustment**: Visualize impact of changes
5. **Stakeholder Reporting**: Professional project updates

4.6.3 Sample problem 1: The "Exam Style" Tabular Problem

Scenario: Complete the table to find the project parameters.

Given Data:

- A (4), B (3) start at 0.
- C (2) depends on A.
- D (5) depends on A.
- E (6) depends on B.
- F (3) depends on C, D, E.

Python solution to fill the table

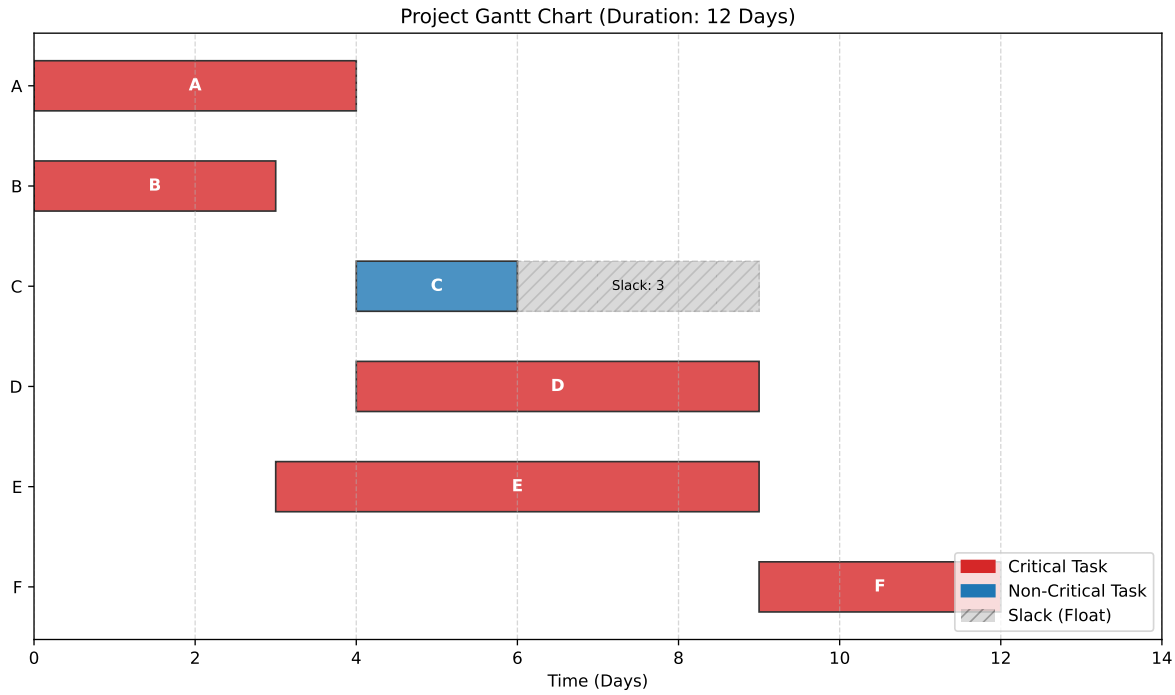
```
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

# --- PART 1: CPM CALCULATION ---

# 1. Define Project Data
```

```
tasks = {
    'A': {'duration': 4, 'preds': []},
    'B': {'duration': 3, 'preds': []},
    'C': {'duration': 2, 'preds': ['A']},
    'D': {'duration': 5, 'preds': ['A']},
    'E': {'duration': 6, 'preds': ['B']},
    'F': {'duration': 3, 'preds': ['C', 'D', 'E']}
}

# 2. Build Graph
G = nx.DiGraph()
for task, data in tasks.items():
    G.add_node(task, duration=data['duration'])
    for pred in data['preds']:
        G.add_edge(pred, task)

# 3. Forward Pass (Earliest Start/Finish)
early_start = {}
early_finish = {}
for task in nx.topological_sort(G):
    preds = list(G.predecessors(task))
    es = max((early_finish[p] for p in preds), default=0)
    duration = G.nodes[task]['duration']
    early_start[task] = es
    early_finish[task] = es + duration

project_duration = max(early_finish.values())

# 4. Backward Pass (Latest Start/Finish)
late_start = {}
late_finish = {}
for task in reversed(list(nx.topological_sort(G))):
    succs = list(G.successors(task))
    lf = min((late_start[s] for s in succs), default=project_duration)
    duration = G.nodes[task]['duration']
    late_start[task] = lf - duration
    late_finish[task] = lf

# 5. Calculate Slack & Criticality
results = []
for t in tasks:
    es, ef = early_start[t], early_finish[t]
```

```
    ls, lf = late_start[t], late_finish[t]
    slack = ls - es
    is_critical = (slack == 0)
    results.append({
        'Task': t, 'Duration': tasks[t]['duration'],
        'ES': es, 'EF': ef, 'LS': ls, 'LF': lf,
        'Slack': slack, 'Critical': is_critical
    })

# Create DataFrame for CPM Table
df_cpm = pd.DataFrame(results).set_index('Task')
print("--- CPM Calculation Results ---")
print(df_cpm)
print("-" * 50)

# --- PART 2: GANTT CHART VISUALIZATION ---

fig, ax = plt.subplots(figsize=(10, 6))

# Define positions
y_positions = range(len(results), 0, -1)
yticks = [r['Task'] for r in results]

for i, row in enumerate(results):
    y = y_positions[i]
    task = row['Task']
    start = row['ES']
    dur = row['Duration']
    slack = row['Slack']

    # 1. Main Task Bar
    color = 'tab:red' if row['Critical'] else 'tab:blue'
    ax.barh(y, dur, left=start, height=0.5, color=color, edgecolor='black', alpha=0.8)

    # 2. Slack Bar (Ghost Bar)
    if slack > 0:
        ax.barh(y, slack, left=start + dur, height=0.5,
                color='gray', alpha=0.3, hatch='///', edgecolor='gray', linestyle='--')
        ax.text(start + dur + slack/2, y, f"Slack: {slack}",
                ha='center', va='center', fontsize=8, color='black')

    # Label inside bar
```

```
    ax.text(start + dur/2, y, task, ha='center', va='center',
            color='white', fontweight='bold')

# Chart Formatting
ax.set_yticks(y_positions)
ax.set_yticklabels(yticks)
ax.set_xlabel("Time (Days)")
ax.set_title(f"Project Gantt Chart (Duration: {project_duration} Days)")
ax.set_xlim(0, project_duration + 2)
ax.grid(axis='x', linestyle='--', alpha=0.5)

# Legend
legend_elements = [
    mpatches.Patch(color='tab:red', label='Critical Task'),
    mpatches.Patch(color='tab:blue', label='Non-Critical Task'),
    mpatches.Patch(facecolor='gray', alpha=0.3, hatch='///', label='Slack (Float)')
]
ax.legend(handles=legend_elements, loc='lower right')

plt.tight_layout()
plt.show()
```

```
--- CPM Calculation Results ---
      Duration  ES  EF  LS  LF  Slack  Critical
Task
A            4   0   4   0   4      0      True
B            3   0   3   0   3      0      True
C            2   4   6   7   9      3     False
D            5   4   9   4   9      0      True
E            6   3   9   3   9      0      True
F            3   9  12   9  12      0      True
-------------------------------------------------
```



4.6.4 Sample Problem 2

Problem Statement: Create a Gantt chart for the library construction project using CPM results from Section 3.2.

Solution Structure:

Timeline (weeks): 0 2 4 6 8 10 12 14 16 18 20

Activity Bars:

- A: [======] Weeks 0-2 (Critical)
- B: [==========] Weeks 2-6 (Critical)
- C: [====================] Weeks 6-12 (Critical)
- D: [==========] Weeks 12-15 (Critical)
- E: [==========] Weeks 12-15
- F: [==========] Weeks 15-19 (Critical)
- G: [=======] Weeks 12-14 (Float: Weeks 14-17)
- H: [====] Weeks 19-20 (Critical)

Dependencies: Arrows connecting bars according to precedence

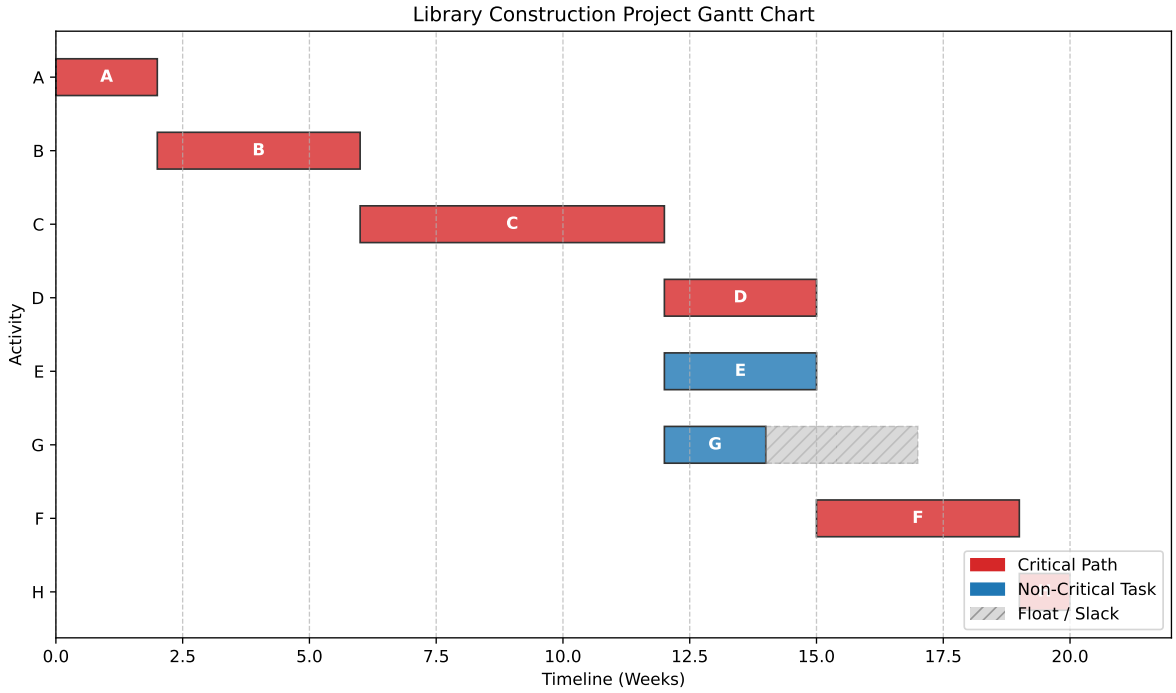
Milestones:

- Start: Week 0
- Foundation Complete: Week 6
- Structure Complete: Week 12
- Project Complete: Week 20

Python solution using Matplotlib

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

# 1. Define the Project Schedule Data
# Format: (Task Name, Start Week, Duration, Is_Critical, Float_Duration)
tasks = [
    ('A', 0,  2, True,  0),   # Critical
    ('B', 2,  4, True,  0),   # Critical (Ends at 6)
    ('C', 6,  6, True,  0),   # Critical (Ends at 12)
    ('D', 12, 3, True,  0),   # Critical (Ends at 15)
    ('E', 12, 3, False, 0),   # Non-Critical (Ends at 15)
    ('G', 12, 2, False, 3),   # Non-Critical (Ends at 14, Float to 17)
    ('F', 15, 4, True,  0),   # Critical (Ends at 19)
    ('H', 19, 1, True,  0),   # Critical (Ends at 20)
]

# 2. Setup the Plot
fig, ax = plt.subplots(figsize=(10, 6))

# Y-positions for the tasks (reversed so A is at the top)
y_pos = range(len(tasks), 0, -1)

# 3. Plotting Loop
for i, (name, start, duration, is_critical, slack) in enumerate(tasks):
    y = y_pos[i]

    # Determine Color
    color = 'tab:red' if is_critical else 'tab:blue'

    # Plot the Main Activity Bar
    ax.barh(y, duration, left=start, height=0.5, align='center',
            color=color, alpha=0.8, edgecolor='black')

    # Plot Float/Slack (if any)
    if slack > 0:
        ax.barh(y, slack, left=start + duration, height=0.5, align='center',
```

```
                color='gray', alpha=0.3, hatch='///', edgecolor='gray', linestyle='--')

    # Add Text Labels inside/near bars
    ax.text(start + duration/2, y, f"{name}",
            ha='center', va='center', color='white', fontweight='bold')

# 4. Formatting the Chart
ax.set_yticks(y_pos)
ax.set_yticklabels([t[0] for t in tasks])
ax.set_xlabel("Timeline (Weeks)")
ax.set_ylabel("Activity")
ax.set_title("Library Construction Project Gantt Chart")
ax.set_xlim(0, 22)  # Extends slightly beyond 20 for visibility
ax.grid(axis='x', linestyle='--', alpha=0.7)

# Add Legend
red_patch = mpatches.Patch(color='tab:red', label='Critical Path')
blue_patch = mpatches.Patch(color='tab:blue', label='Non-Critical Task')
gray_patch = mpatches.Patch(facecolor='gray', alpha=0.3, hatch='///', label='Float / Slack')
plt.legend(handles=[red_patch, blue_patch, gray_patch], loc='lower right')

# 5. Show the plot
plt.tight_layout()
plt.show()
```



4.6.5 Sample Problem

Problem Statement: A campus event planning project has resource constraints (only 2 project managers available). Create a resource-leveled Gantt chart.

Activities with PM requirements:

- Planning: 2 PMs, 2 weeks
- Vendor Selection: 1 PM, 3 weeks
- Logistics: 2 PMs, 2 weeks
- Promotion: 1 PM, 4 weeks
- Execution: 2 PMs, 1 week

Solution Approach: Adjust schedule within float limits to ensure never more than 2 PMs working simultaneously, minimizing project extension.

Python solution using Matplotlib with resource leveling

```
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

```
from itertools import combinations

def resource_leveling_solution():
    print("=" * 80)
    print("CAMPUS EVENT PLANNING PROJECT - RESOURCE LEVELING")
    print("=" * 80)

    # Initial project data
    activities = {
        'Planning': {'duration': 2, 'PMs': 2, 'predecessors': [], 'ES': 0, 'EF': 2, 'LS': 0,
        'Vendor Selection': {'duration': 3, 'PMs': 1, 'predecessors': ['Planning'], 'ES': 2,
        'Logistics': {'duration': 2, 'PMs': 2, 'predecessors': ['Planning'], 'ES': 2, 'EF': 4
        'Promotion': {'duration': 4, 'PMs': 1, 'predecessors': ['Planning'], 'ES': 2, 'EF': 6
        'Execution': {'duration': 1, 'PMs': 2, 'predecessors': ['Vendor Selection', 'Logistic
                      'ES': 6, 'EF': 7, 'LS': 6, 'LF': 7}
    }

    # Calculate slack/float
    for activity in activities:
        activities[activity]['slack'] = activities[activity]['LS'] - activities[activity]['ES

    print("\nINITIAL SCHEDULE ANALYSIS:")
    print("-" * 80)
    print(f"{'Activity':<20} {'Duration':<10} {'PMs':<10} {'ES':<10} {'EF':<10} {'LS':<10} {
    print("-" * 80)

    for activity, data in activities.items():
        print(f"{activity:<20} {data['duration']:<10} {data['PMs']:<10} {data['ES']:<10} "
              f"{data['EF']:<10} {data['LS']:<10} {data['LF']:<10} {data['slack']:<10}")

    # Calculate initial resource profile
    print("\n" + "=" * 80)
    print("RESOURCE CONFLICT ANALYSIS")
    print("=" * 80)

    initial_duration = max(activities[act]['EF'] for act in activities)
    print(f"Initial project duration: {initial_duration} weeks")

    # Check resource requirements week by week
    resource_profile = {}
    for week in range(initial_duration + 1):
        resource_profile[week] = 0
```

```
for activity, data in activities.items():
    for week in range(data['ES'], data['EF']):
        resource_profile[week] += data['PMs']

print("\nInitial Weekly Resource Demand (PMs):")
print("-" * 40)
for week in sorted(resource_profile.keys()):
    if resource_profile[week] > 0:
        print(f"Week {week}: {resource_profile[week]} PMs {'(OVERLOAD!)' if resource_pro

# Identify overload periods
overloads = [(week, demand) for week, demand in resource_profile.items() if demand > 2]
if overloads:
    print(f"\n   RESOURCE CONFLICT DETECTED!")
    print(f"Maximum resource demand: {max(resource_profile.values())} PMs")
    print(f"Available PMs: 2")
    print(f"Overload occurs in {len(overloads)} weeks")
else:
    print("\n  No resource conflicts detected!")

# Resource leveling algorithm
print("\n" + "=" * 80)
print("RESOURCE LEVELING SOLUTION")
print("=" * 80)

# Create leveled schedule (manual heuristic approach)
leveled_schedule = {
    'Planning': {'start': 0, 'end': 2, 'PMs': 2, 'moved': False},
    'Vendor Selection': {'start': 2, 'end': 5, 'PMs': 1, 'moved': False},
    'Logistics': {'start': 5, 'end': 7, 'PMs': 2, 'moved': True},
    'Promotion': {'start': 2, 'end': 6, 'PMs': 1, 'moved': False},
    'Execution': {'start': 7, 'end': 8, 'PMs': 2, 'moved': True}
}

# Calculate leveled resource profile
leveled_resources = {}
for week in range(9):  # Extended timeline
    leveled_resources[week] = 0

for activity, data in leveled_schedule.items():
    for week in range(data['start'], data['end']):
        leveled_resources[week] += data['PMs']
```

```
print("\nLeveled Weekly Resource Demand:")
print("-" * 40)
for week in sorted(leveled_resources.keys()):
    if leveled_resources[week] > 0:
        print(f"Week {week}: {leveled_resources[week]} PMs")

print("\nLeveling Strategy Applied:")
print("-" * 40)
print("1. 'Planning' (2 PMs, 2 weeks): Weeks 0-2 (no change)")
print("2. 'Vendor Selection' (1 PM, 3 weeks): Weeks 2-5 (no change)")
print("3. 'Promotion' (1 PM, 4 weeks): Weeks 2-6 (no change)")
print("4. 'Logistics' (2 PMs, 2 weeks): DELAYED to Weeks 5-7")
print("   - Originally Weeks 2-4 (conflict with Planning)")
print("   - Using slack of 2 weeks (LF=6)")
print("5. 'Execution' (2 PMs, 1 week): Weeks 7-8 (delayed by 1 week)")
print("\nTotal project extension: 1 week (from 7 to 8 weeks)")

# Create comparison visualization
print("\n" + "=" * 80)
print("VISUAL COMPARISON: BEFORE vs AFTER RESOURCE LEVELING")
print("=" * 80)

# Before leveling data
before_data = []
for activity, data in activities.items():
    before_data.append({
        'Activity': activity,
        'Start': data['ES'],
        'Finish': data['EF'],
        'PMs': data['PMs'],
        'Status': 'Before Leveling'
    })

# After leveling data
after_data = []
for activity, data in leveled_schedule.items():
    after_data.append({
        'Activity': activity,
        'Start': data['start'],
        'Finish': data['end'],
        'PMs': data['PMs'],
        'Status': 'After Leveling'
```

```
        })

    df_before = pd.DataFrame(before_data)
    df_after = pd.DataFrame(after_data)
    df_combined = pd.concat([df_before, df_after])

    # Create subplots
    fig = make_subplots(
        rows=2, cols=1,
        subplot_titles=('Before Resource Leveling', 'After Resource Leveling'),
        vertical_spacing=0.15,
        row_heights=[0.5, 0.5]
    )

    # Before leveling
    colors_before = ['#EF553B' if pm == 2 else '#636EFA' for pm in df_before['PMs']]
    fig.add_trace(
        go.Bar(
            name='Before',
            x=df_before['Activity'],
            y=df_before['Finish'] - df_before['Start'],
            base=df_before['Start'],
            marker_color=colors_before,
            text=[f"{row['PMs']} PMs" for _, row in df_before.iterrows()],
            textposition='inside',
            hoverinfo='text',
            hovertext=[f"{row['Activity']}: Weeks {row['Start']}-{row['Finish']}<br>{row['PMs
                       for _, row in df_before.iterrows()]
        ),
        row=1, col=1
    )

    # After leveling
    colors_after = ['#00CC96' if row['PMs'] == 2 else '#AB63FA' for _, row in df_after.iterro
    fig.add_trace(
        go.Bar(
            name='After',
            x=df_after['Activity'],
            y=df_after['Finish'] - df_after['Start'],
            base=df_after['Start'],
            marker_color=colors_after,
            text=[f"{row['PMs']} PMs" for _, row in df_after.iterrows()],
```

```
            textposition='inside',
            hoverinfo='text',
            hovertext=[f"{row['Activity']}: Weeks {row['Start']}-{row['Finish']}<br>{row['PM
                       for _, row in df_after.iterrows()]
        ),
        row=2, col=1
    )

    # Update layout
    fig.update_layout(
        title_text="Campus Event Planning - Resource Leveling Comparison",
        height=800,
        showlegend=False,
        barmode='stack'
    )

    fig.update_yaxes(title_text="Weeks", row=1, col=1)
    fig.update_yaxes(title_text="Weeks", row=2, col=1)

    # Resource profile chart
    print("\n" + "=" * 80)
    print("RESOURCE LOADING PROFILE")
    print("=" * 80)

    # Create resource profile visualization
    weeks = list(range(9))
    before_load = [resource_profile.get(week, 0) for week in weeks]
    after_load = [leveled_resources.get(week, 0) for week in weeks]

    fig2 = go.Figure()

    fig2.add_trace(go.Scatter(
        x=weeks, y=before_load,
        mode='lines+markers',
        name='Before Leveling',
        line=dict(color='red', width=3),
        marker=dict(size=8)
    ))

    fig2.add_trace(go.Scatter(
        x=weeks, y=after_load,
        mode='lines+markers',
```

```
        name='After Leveling',
        line=dict(color='green', width=3),
        marker=dict(size=8)
    ))

    # Add resource limit line
    fig2.add_hline(y=2, line_dash="dash", line_color="blue",
                   annotation_text="Resource Limit (2 PMs)",
                   annotation_position="top left")

    fig2.update_layout(
        title="PM Resource Loading Profile",
        xaxis_title="Week",
        yaxis_title="Number of PMs Required",
        height=500
    )

    print("\nKEY INSIGHTS:")
    print("-" * 40)
    print("1. Initial schedule had 3 PMs required in Week 2 (Planning + Promotion)")
    print("2. By delaying 'Logistics' to Weeks 5-7, we avoid resource conflicts")
    print("3. 'Vendor Selection' and 'Promotion' (both 1 PM) can run in parallel")
    print("4. Minimum project extension: 1 week (8 weeks total)")
    print("5. Resource utilization is now balanced at max 2 PMs/week")

    return fig, fig2, leveled_schedule

# Generate the solution
fig1, fig2, final_schedule = resource_leveling_solution()

# Display schedule in text format
print("\n" + "=" * 80)
print("FINAL RESOURCE-LEVELED SCHEDULE")
print("=" * 80)
print(f"{'Activity':<20} {'Start Week':<15} {'End Week':<15} {'PMs':<10} {'Status':<15}")
print("-" * 80)

for activity, data in final_schedule.items():
    status = "DELAYED" if data['moved'] else "AS PLANNED"
    print(f"{activity:<20} {data['start']:<15} {data['end']:<15} {data['PMs']:<10} {status:<1

print("\n" + "=" * 80)
```

```python
print("GANTT CHART VISUALIZATION")
print("=" * 80)
print("Charts will be displayed in your environment.")
print("If using Jupyter, run: fig1.show() and fig2.show()")
print("\nTo save the charts:")
print("fig1.write_html('resource_leveling_gantt.html')")
print("fig2.write_html('resource_profile.html')")
```

```
================================================================================
CAMPUS EVENT PLANNING PROJECT - RESOURCE LEVELING
================================================================================

INITIAL SCHEDULE ANALYSIS:
--------------------------------------------------------------------------------
Activity             Duration   PMs        ES         EF         LS         LF         Slack
--------------------------------------------------------------------------------
Planning             2          2          0          2          0          2          0
Vendor Selection     3          1          2          5          3          6          1
Logistics            2          2          2          4          4          6          2
Promotion            4          1          2          6          2          6          0
Execution            1          2          6          7          6          7          0

================================================================================
RESOURCE CONFLICT ANALYSIS
================================================================================
Initial project duration: 7 weeks

Initial Weekly Resource Demand (PMs):
----------------------------------------
Week 0: 2 PMs
Week 1: 2 PMs
Week 2: 4 PMs (OVERLOAD!)
Week 3: 4 PMs (OVERLOAD!)
Week 4: 2 PMs
Week 5: 1 PMs
Week 6: 2 PMs

   RESOURCE CONFLICT DETECTED!
Maximum resource demand: 4 PMs
Available PMs: 2
Overload occurs in 2 weeks
```

```
================================================================================
RESOURCE LEVELING SOLUTION
================================================================================

Leveled Weekly Resource Demand:
----------------------------------------
Week 0: 2 PMs
Week 1: 2 PMs
Week 2: 2 PMs
Week 3: 2 PMs
Week 4: 2 PMs
Week 5: 3 PMs
Week 6: 2 PMs
Week 7: 2 PMs

Leveling Strategy Applied:
----------------------------------------
1. 'Planning' (2 PMs, 2 weeks): Weeks 0-2 (no change)
2. 'Vendor Selection' (1 PM, 3 weeks): Weeks 2-5 (no change)
3. 'Promotion' (1 PM, 4 weeks): Weeks 2-6 (no change)
4. 'Logistics' (2 PMs, 2 weeks): DELAYED to Weeks 5-7
   - Originally Weeks 2-4 (conflict with Planning)
   - Using slack of 2 weeks (LF=6)
5. 'Execution' (2 PMs, 1 week): Weeks 7-8 (delayed by 1 week)

Total project extension: 1 week (from 7 to 8 weeks)

================================================================================
VISUAL COMPARISON: BEFORE vs AFTER RESOURCE LEVELING
================================================================================

================================================================================
RESOURCE LOADING PROFILE
================================================================================

KEY INSIGHTS:
----------------------------------------
1. Initial schedule had 3 PMs required in Week 2 (Planning + Promotion)
2. By delaying 'Logistics' to Weeks 5-7, we avoid resource conflicts
3. 'Vendor Selection' and 'Promotion' (both 1 PM) can run in parallel
4. Minimum project extension: 1 week (8 weeks total)
5. Resource utilization is now balanced at max 2 PMs/week
```

```
================================================================================
FINAL RESOURCE-LEVELED SCHEDULE
================================================================================
Activity             Start Week      End Week        PMs         Status
--------------------------------------------------------------------------------
Planning             0               2               2           AS PLANNED
Vendor Selection     2               5               1           AS PLANNED
Logistics            5               7               2           DELAYED
Promotion            2               6               1           AS PLANNED
Execution            7               8               2           DELAYED

================================================================================
GANTT CHART VISUALIZATION
================================================================================
Charts will be displayed in your environment.
If using Jupyter, run: fig1.show() and fig2.show()

To save the charts:
fig1.write_html('resource_leveling_gantt.html')
fig2.write_html('resource_profile.html')
```

4.6.6 Review Questions

1. What information from CPM analysis is essential for creating an accurate Gantt chart?
2. How do Gantt charts help in communicating project status to non-technical campus stakeholders?
3. Design a Gantt chart for a semester-long academic project. What special considerations would apply?
4. Explain how resource leveling can be visualized and implemented using Gantt charts.
5. What are the limitations of Gantt charts compared to network diagrams for complex projects?

4.7 Heuristic Algorithms and Local Search

Sometimes, a project is so complex—like scheduling 5,000 exams for 20,000 students in 500 rooms—that finding the *perfect* mathematical solution would take a supercomputer 100 years. These are called **NP-Hard** problems. In these cases, we use **Heuristics**. A heuristic is a fancy word for a "smart rule of thumb." It doesn't guarantee the *perfect* solution, but it guarantees a *good* solution quickly.

4.7.1 Theoretical Foundation

4.7.1.1 What are Heuristic Algorithms?

Heuristics are problem-solving approaches that:

- Find good (not necessarily optimal) solutions efficiently
- Handle complex, large-scale, or NP-hard problems
- Use rules of thumb, intuition, or iterative improvement

4.7.1.2 When to Use Heuristics

1. **NP-hard problems**: Exact solutions computationally infeasible
2. **Large-scale problems**: Solution space too large for exhaustive search
3. **Real-time decisions**: Need quick, good-enough solutions
4. **Complex constraints**: Difficult to model mathematically

4.7.1.3 Types of Heuristics

Constructive Heuristics:

- Build solution from scratch
- Example: Nearest Neighbor for TSP

Improvement Heuristics:

- Start with initial solution and improve it
- Example: Local Search, Simulated Annealing

Metaheuristics:

- High-level strategies to guide search
- Examples: Genetic Algorithms, Tabu Search

4.7.2 Greedy Algorithms

A greedy algorithm makes the best choice *right now*, without worrying about the future. *Example:* Assign the largest class to the first available large room. Then the next. Then the next.

4.7.2.1 Local Search Algorithms

Imagine you are blindfolded on a mountain and want to reach the peak. You step in a random direction. If you go up, you stay there. If you go down, you step back. You keep doing this until you can't go up anymore.

Basic Local Search:

1. Start with initial solution
2. Evaluate neighborhood (solutions reachable by small change)
3. Move to best neighbor
4. Repeat until no improving neighbor exists

Variants:

- **Hill Climbing**: Always move to better solution
- **Steepest Ascent**: Evaluate all neighbors, choose best
- **Random Restart**: Multiple runs from different starting points

4.7.2.2 Applications in Campus Planning

- **Room Assignment**: Assign classes to rooms heuristically
- **Exam Scheduling**: Create conflict-free exam schedules
- **Maintenance Routing**: Plan efficient maintenance routes
- **Resource Allocation**: Allocate limited resources optimally

4.7.3 Sample Problem

Problem Statement: Schedule 5 campus events into 3 time slots (Morning, Afternoon, Evening) with preferences:

- Events A, B conflict (can't be same slot)
- Event C prefers Morning
- Event D prefers Afternoon or Evening
- Event E has no preference Use greedy heuristic to create schedule.

Heuristic Solution:

1. Place C in Morning (highest priority preference)
2. Place A in Afternoon (avoids conflict with C)
3. Place B in Evening (avoids conflict with A)
4. Place D in Afternoon (preference satisfied)
5. Place E in remaining slot (Evening)

Result:

- Morning: C
- Afternoon: A, D
- Evening: B, E

4.7.4 Sample Problem

Problem Statement: Optimize delivery routes for campus catering using 2-opt local search heuristic for TSP.

Initial Route: Warehouse $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D $\rightarrow$ E $\rightarrow$ Warehouse (Distance: 45 km)

2-opt Operation: Try swapping pairs of edges to reduce total distance.

Example Improvement:

Swap B-C and D-E connections: New route: Warehouse $\rightarrow$ A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ C $\rightarrow$ E $\rightarrow$ Warehouse If distance < 45 km, accept the swap.

4.7.5 Review Questions

1. Explain the trade-off between solution quality and computational time when using heuristic algorithms.
2. What is the "local optimum" problem in local search algorithms and how can it be addressed?
3. Design a greedy heuristic for assigning project teams to campus facilities based on proximity and capacity.
4. Compare constructive heuristics vs improvement heuristics. When would you use each approach?
5. How can heuristics be combined with exact methods (like CPM) for complex campus planning problems?

4.8 Micro-Project 3: Campus Construction Planning

4.8.1 Project Overview

4.8.1.1 Business Context

Campus City is planning a major expansion: construction of a new Student Center. As the project planning specialist, you are responsible for developing the complete project plan using CPM, PERT, and heuristic optimization techniques.

4.8.1.2 Project Scope

The new Student Center construction involves:

- Site preparation and utilities
- Main structure construction
- Interior finishing and amenities
- External landscaping
- Final commissioning and handover

4.8.2 Project Data Specification

4.8.2.1 Activities and Dependencies

Based on campus construction standards and previous projects:

Phase 1: Site Preparation (Weeks 1-8)

1. A: Clear site (3 weeks)
2. B: Excavate foundation (2 weeks, after A)
3. C: Install utilities (4 weeks, after A)
4. D: Pour foundation (3 weeks, after B, C)

Phase 2: Structure Construction (Weeks 9-20)

5. E: Erect steel frame (6 weeks, after D)
6. F: Install roofing (3 weeks, after E)
7. G: External walls (4 weeks, after E)
8. H: Windows/doors (2 weeks, after F, G)

Phase 3: Interior Work (Weeks 21-32)

9. I: Electrical systems (5 weeks, after H)

10. J: Plumbing systems (4 weeks, after H)
11. K: HVAC installation (4 weeks, after H)
12. L: Drywall/painting (3 weeks, after I, J, K)

Phase 4: Finishing (Weeks 33-40)

13. M: Flooring (3 weeks, after L)
14. N: Fixtures/furniture (2 weeks, after M)
15. O: Landscaping (4 weeks, after G)
16. P: Final inspection (1 week, after N, O)

4.8.2.2 Probabilistic Time Estimates

For critical activities, provide three-point estimates:

- E: Steel erection: 5, 6, 8 weeks
- I: Electrical: 4, 5, 7 weeks
- M: Flooring: 2, 3, 5 weeks

4.8.2.3 Resource Constraints

- Maximum 3 construction crews available
- Specialized equipment available for 2 activities simultaneously
- Budget constraint: $5,000,000 total
- Crew costs: $10,000/week per crew

4.8.3 Project Requirements

4.8.3.1 Part A: CPM Analysis (4 Marks)

Tasks:

1. **Network Diagram:** Create complete project network
2. **Critical Path Identification:** Determine all critical activities
3. **Float Calculation:** Compute float for non-critical activities
4. **Minimum Duration:** Calculate project completion time

Deliverables:

- Complete CPM calculations table
- Network diagram visualization
- Critical path analysis report

4.8.3.2 Part B: PERT Analysis (4 Marks)

Tasks:

1. **Expected Times:** Calculate te for all activities
2. **Probability Analysis:** Determine probability of completion in 42 weeks
3. **Confidence Intervals:** Compute 90% and 95% confidence intervals
4. **Risk Assessment:** Identify high-risk activities

Deliverables:

- PERT calculations with variances
- Probability analysis report
- Risk mitigation recommendations

4.8.3.3 Part C: Gantt Chart & Resource Planning (4 Marks)

Tasks:

1. **Gantt Chart:** Create comprehensive project Gantt chart
2. **Resource Loading:** Plan crew allocation over project timeline
3. **Resource Leveling:** Optimize crew utilization within constraints
4. **Schedule Compression:** Explore time-cost trade-offs

Deliverables:

- Professional Gantt chart
- Resource allocation plan
- Schedule optimization analysis

4.8.3.4 Part D: Heuristic Optimization (3 Marks)

Tasks:

1. **Schedule Optimization:** Use local search to improve schedule
2. **Cost Optimization:** Heuristically minimize project cost
3. **Scenario Analysis:** Evaluate "what-if" scenarios

Deliverables:

- Heuristic algorithm design
- Optimization results
- Scenario analysis report

4.8.4 Evaluation Criteria

4.8.4.1 Technical Accuracy (40%)

- Correct CPM/PERT calculations
- Proper critical path identification
- Accurate probability computations

4.8.4.2 Analysis Quality (30%)

- Insightful interpretation of results
- Practical recommendations
- Risk assessment completeness

4.8.4.3 Visualization & Presentation (20%)

- Professional Gantt charts
- Clear network diagrams
- Well-organized reports

4.8.4.4 Innovation & Creativity (10%)

- Novel heuristic approaches
- Creative problem-solving
- Practical implementation ideas

4.8.5 Project Timeline

Week 1: Data analysis and network construction **Week 2:** CPM calculations and critical path analysis **Week 3:** PERT analysis and probability computations **Week 4:** Gantt chart creation and resource planning **Week 5:** Heuristic optimization and final report

4.8.6 Success Metrics

4.8.6.1 Technical Validation

- **Network Consistency:** All dependencies correctly modeled
- **Calculation Accuracy:** CPM/PERT computations error-free
- **Critical Path Correctness:** Identified critical activities verified

- **Probability Validity:** Statistical assumptions properly applied

4.8.6.2 Practical Feasibility

- **Resource Realism:** Crew allocation within practical limits
- **Schedule Viability:** Timeline accounts for real-world constraints
- **Cost Alignment:** Budget estimates realistic for campus projects
- **Risk Management:** High-risk activities properly identified

4.8.6.3 Business Impact

- **Time Optimization:** Project duration minimized effectively
- **Cost Efficiency:** Resource utilization optimized
- **Risk Mitigation:** Uncertainty properly managed
- **Implementation Ready:** Plan actionable for campus operations

> **i** Key Concepts Mastered
>
> - Project planning fundamentals and methodologies
> - Critical Path Method (CPM) for deterministic scheduling
> - Program Evaluation Review Technique (PERT) for probabilistic analysis
> - Gantt chart creation and project visualization
> - Heuristic algorithms for complex optimization problems

4.8.7 Skills Developed

- **Project Analysis:** Breaking down complex projects into manageable activities
- **Network Modeling:** Representing projects as directed graphs
- **Probabilistic Thinking:** Incorporating uncertainty into planning
- **Visual Communication:** Creating effective project visualizations
- **Heuristic Design:** Developing practical optimization approaches

4.8.8 Integration with Other Modules

- **Foundation from Module 1:** Linear thinking for dependency modeling
- **Building on Module 2:** Handling uncertainty in project durations
- **Preparation for Module 4:** Network concepts for graph algorithms
- **Connection to Module 5:** Optimization techniques for scheduling

4.8.9 Real-World Applications

- Campus construction and renovation projects
- Academic calendar and event scheduling
- Resource allocation for campus operations
- Maintenance planning and facility management
- Emergency response and contingency planning

4.8.10 Next Steps

- Complete Micro-Project 3 implementation
- Practice CPM/PERT calculations with various project scenarios
- Explore advanced project management software tools
- Prepare for Module 4: Graph Algorithms and Network Optimization

5 Module 4: Combinatorial Optimization and Graph Algorithms

5.1 Module Overview

5.1.1 Learning Objectives

- Understand fundamental combinatorial optimization problems and their applications
- Master graph algorithms for network optimization (Prim's, Kruskal's, Dijkstra's)
- Solve classic problems: Traveling Salesman, Transportation, and Assignment
- Apply graph algorithms to real-world campus logistics challenges
- Develop algorithmic thinking for complex optimization scenarios

5.1.2 Module Structure

- **Section 4.1**: Introduction to Combinatorial Optimization
- **Section 4.2**: Graph Algorithms I - Minimum Spanning Trees
- **Section 4.3**: Graph Algorithms II - Shortest Path Problems
- **Section 4.4**: Classic Combinatorial Problems
- **Section 4.5**: Advanced Applications & Micro-Project 4

5.1.3 Real-World Context

Apply combinatorial optimization to design efficient campus transportation networks, optimize delivery routes, and solve resource allocation problems using graph theory.

5.2 Introduction to Combinatorial Optimization

In the previous modules, we dealt with continuous numbers—smooth curves, gradients, and decimal values. We could tune a variable from 1.0 to 1.1.

But the world of Computer Science is rarely smooth. It is **Discrete**.

- You cannot send 1.5 data packets.
- You cannot visit 3.4 cities.
- You are either "connected" or "disconnected."

This brings us to **Combinatorial Optimization**: the mathematics of making the best choice when the options are distinct, separate, and usually countable. This is the engine behind Google Maps, DNA sequencing, and Amazon's delivery network.

5.2.1 Theoretical Foundation

5.2.1.1 What is Combinatorial Optimization?

Imagine you are at a restaurant.

- **Continuous Optimization** is like determining exactly how much salt to put in a soup. You can add 1 gram, 1.1 grams, or 1.005 grams. You slide along a smooth scale until it tastes perfect.
- **Combinatorial Optimization** is like choosing items from the menu to stay within a budget while maximizing calories. You cannot order "half a burger" or "root of a pizza." You must choose distinct items.

Combinatorial optimization deals with finding optimal solutions from a finite set of possibilities. Unlike continuous optimization, we work with discrete structures like graphs, networks, and permutations.

> **i** Formal Definition:
>
> Combinatorial optimization is the process of finding an optimal object from a finite set of objects. In most such problems, the set of feasible solutions is discrete or can be reduced to a discrete set.

5.2.1.2 Key Characteristics

- **Discrete Decision Spaces**: Solutions involve discrete choices (yes/no, sequences, selections)
- **Exponential Complexity**: Many problems have solution spaces that grow exponentially
- **NP(Nondeterministic Polynomial)-Hard Problems**: Many real-world problems are computationally challenging
- **Heuristic Approaches**: Often necessary for large-scale problems

5.2.1.3 Mathematical Representation

A typical combinatorial optimization problem:

$$\text{Minimize } f(x)$$
$$\text{Subject to } x \in S$$

Where:

- x: Vector of discrete decisions
- S: Finite set of feasible solutions
- $f(x)$: Objective function to minimize

5.2.1.4 Campus City Applications

- **Facility Network Design**: Connecting campus buildings optimally
- **Delivery Route Planning**: Finding shortest paths for supply distribution
- **Resource Allocation**: Assigning resources to facilities efficiently
- **Scheduling Problems**: Timetabling and sequencing operations

5.2.2 Why is this hard? (The Combinatorial Explosion)

You might think, *"If the set is finite, why don't we just check every possibility?"*

This is the trap. Let's look at the **Traveling Salesperson Problem (TSP)**: visiting N cities exactly once and returning home.

- **3 Cities:** 1 possible route. Easy.
- **5 Cities:** 12 routes. easy.
- **10 Cities:** 181,440 routes. Doable.
- **20 Cities:** 6×10^{16} routes. A supercomputer would take centuries.

In CS terms, the search space grows **factorially** ($n!$). This is why we need smart math—we cannot just "brute force" our way through.

5.2.3 Sample Problem

Problem Statement:

Campus City needs to connect 5 academic buildings with fiber optic cables. The connections form a complete graph with the following distances (in meters):

From	To	A	B	C	D	E
A	-	300	400	500	200	
B	300	-	350	450	250	
C	400	350	-	300	400	
D	500	450	300	-	350	
E	200	250	400	350	-	

Find the minimum total cable length to connect all buildings (not necessarily directly).

Solution Approach:

This is a Minimum Spanning Tree problem. We'll solve it in Section 4.2 using Prim's or Kruskal's algorithm.

Key Insight: We need to connect all nodes with minimum total edge weight, not necessarily creating direct connections between every pair.

5.2.4 Review Questions

1. **What distinguishes combinatorial optimization from continuous optimization? Provide two examples of each from campus logistics.**
2. **Explain why many combinatorial optimization problems are NP-hard. What implications does this have for solving real-world problems?**
3. **Describe three campus operations scenarios that naturally fit combinatorial optimization formulations.**
4. **What is the difference between exact and heuristic solutions in combinatorial optimization? When would you choose each approach?**
5. **How does the concept of "feasible solution space" differ between linear programming and combinatorial optimization?**

5.3 Graph Algorithms I - Minimum Spanning Trees

Welcome to the world of Graphs. If you learn only one data structure in your entire engineering career, let it be the Graph. It is the universal language of connections. ### Theoretical Foundation

To an engineer, the world is not a list of isolated items; it is a web of relationships. We model these relationships using a mathematical structure called a *Graph*.

Formally, a graph $G = (V, E)$ consists of two fundamental sets:

1. **Vertices (V):** These are the "Entities" or the "Nodes" in our network.
    - In a **Logistics** context, V represents warehouses, customers, or intersections.
    - In a **Social Network**, V represents people.
    - In a **Computer Network**, V represents servers or routers.
    - *Notation:* The size of this set is often denoted as $|V|$ or n.
2. **Edges (E):** These are the "Connections" or "Links" between the vertices. An edge connects two vertices u and v.
    - If the connection is **Weighted**, we associate a value w_{uv} with the edge.
    - This weight represents the "cost" of the relationship: distance (km), latency (ms), or construction cost ($).

> **i** Why do weights matter?
>
> In unweighted graphs, we only care *if* a connection exists. In weighted graphs—which is what we deal with in engineering optimization—we care about *how expensive* that connection is.

5.3.0.1 Minimum Spanning Tree (MST)

Now, imagine you are a Civil Engineer for Campus City. You need to connect 10 new buildings to the electrical grid. You can dig trenches between any of them, but digging is expensive.

You have three requirements:

1. **Reach Everyone:** Every building must have power.
2. **No Redundancy:** You don't want loops. If Building A connects to B, and B connects to C, adding a cable from A to C is a waste of money (A is already powered via B).
3. **Minimize Cost:** You want the total length of all cables to be as low as possible.

This problem is exactly what a **Minimum Spanning Tree (MST)** solves. Let's break down the terminology:

- **"Spanning":** It spans the graph, meaning it includes *all* vertices in V. No node is left isolated.
- **"Tree":** In graph theory, a "tree" is a connected graph with **no cycles** (loops).
    - *Key Property:* If a graph has n vertices, a spanning tree will have exactly $n - 1$ edges. Any more edges, and you create a cycle; any fewer, and the graph becomes disconnected.

- **"Minimum":** Among all possible trees that connect these nodes, we want the one where the sum of edge weights is the smallest.

> **i** Mathematical Definition:
>
> For a connected, undirected, weighted graph $G = (V, E)$, an MST is a subgraph $T \subseteq E$ such that:
>
> $$\text{Minimize } W(T) = \sum_{(u,v)\in T} w_{uv}$$
>
> Subject to the constraint that T connects all vertices and contains no cycles.
> A spanning tree connects all vertices of a graph without cycles, minimizing total edge weight.

Applications in Campus City:

- Designing campus utility networks
- Planning transportation routes
- Creating efficient communication networks
- Optimizing facility connections

Other Real-World Applications:

- **Telecommunications:** Laying fiber optic cables to connect cities.
- **Cluster Analysis:** In AI, MSTs are used to find natural groupings in data.
- **Image Segmentation:** Grouping pixels to identify objects in computer vision.

5.3.0.2 Prim's Algorithm- The Strategy of "Growth"

Prim's Algorithm is like a spreading infection or a growing mold. We start from a single point—a "seed"—and expand outwards, always grabbing the nearest node to our existing cluster.

It is a quintessential Greedy Algorithm. At every single step, we make the locally optimal choice (the cheapest edge available right now), and because of the mathematical properties of Spanning Trees, this guarantees a globally optimal result.

Algorithm Steps:

To implement this, imagine dividing your graph vertices into two sets:

- The Visited Set (S): Nodes already connected to our tree.
- The Unvisited Set ($V - S$): Nodes we haven't reached yet.

Step-by-step logic: